Technical Document for "Present Sense"

Ahmed Yehia, Arwaa Mamdoh, Mayar Adel, Mostafa Wael

Supervised by Dr. Fatma Helmy and Eng. Saja Saadoun

Faculty of Computer Science
Misr International University, Cairo, Egypt

Version	Date	Reason for Change
1.0	10-January-2025	Technical Document First version's specifications are defined

Table 1: Document version history

GitHub: `https://github.com/ArwaaMamdoh23/Presentation-Skills.git`

Abstract

The main idea of this project is to use AI technology to help users improve their presentation skills through personalized feedback. By uploading a recorded presentation, users receive expert analysis on tone, body language, pacing, clarity, and facial expressions, along with a personalized rating and improvement suggestions. This allows them to identify weaknesses and refine their delivery for more effective communication. Strong presentation skills are essential for success, yet many struggle to assess their performance. Our AI-powered app bridges this gap by providing structured feedback previously available only through professional coaching. Users simply film and upload their presentation, and the AI evaluates key aspects, offering tailored insights to enhance delivery, build confidence, and make a lasting impact. With its accessibility and real-time analysis, this tool helps individuals continuously improve, making it valuable for students, professionals, and public speakers alike.

Contents

Abstract **2**

1 Introduction **1**

1.1 Motivation 2

1.1.1 Problem Statement 2

1.2 Aims and Objectives 3

1.3 Scope 3

1.4 System Overview 4

1.4.1 Business Context 5

1.4.2 Users Characteristics 5

1.5 Project Management and Deliverables 6

1.5.1 Time Plan 6

1.5.2 Deliverables 7

2 Background and Related Work **8**

2.1 Introduction 8

2.2 Background 8

2.3 Related Systems 9

2.3.1 Academic 9

2.3.2 Business Applications 10

2.4 Summary 11

3 System Requirements Specification **12**

3.1 Introduction 12

3.2 Functional Requirements 12

3.2.1 System Functions 12

3.2.2 Detailed Functional Specification 14
3.3 Interface Requirements 19
3.3.1 User Interfaces 19
3.3.2 Graphical User Interface (GUI) 19
3.3.3 Hardware Interfaces 21
3.3.4 Communication Interfaces 21
3.3.5 Application Programming Interface (API) 22
3.3.6 External APIs 22
3.4 Design Constraints 25
3.4.1 Standards Compliance 25
3.4.2 Software Constraints 25
3.5 Non-functional Requirements 26
3.5.1 Security 26
3.5.2 Availability 26
3.5.3 Maintainability 26
3.5.4 Portability 26
3.5.5 Usability 27
3.5.6 Accessibility 27
3.5.7 Extensibility 27
3.6 Summary 28

4 System Design **29**
4.1 Introduction 29
4.2 Architecture Design viewpoints 29
4.2.1 Context viewpoint 29
4.2.2 Composition viewpoint 30
4.2.3 Logical viewpoint 32
4.2.4 Patterns use viewpoint 34
4.2.5 Algorithm viewpoint 35
4.2.6 Interaction viewpoint 45
4.2.7 Interface viewpoint 49
4.3 Data Design 50
4.3.1 Data Description 50

4.3.2 Dataset Description 53
4.3.3 Database Design 55
4.4 Human Interface Design 56
4.4.1 User Interface 56
4.4.2 Screen Images 56
4.4.3 Screen Objects and Actions 59
4.5 Implementation 59
4.6 Testing Plan 60
4.6.1 Test Scenario X 60
4.6.2 Test Scenario Y 64
4.7 Requirements Matrix 65
4.8 System Deployment 67
4.8.1 Frameworks 67
4.8.2 Tools 68
4.8.3 Technologies 69
4.9 Summary 70

Bibliography **70**

Appendices **75**

A Git Repository **75**

List of Tables

1 Document version history 2

3.1 SingUp Function Description 14
3.2 Log In Function Description 15
3.3 Sign Out Function Description 15
3.4 Edit Profile Function Description 16
3.5 Upload Video Function Description 16
3.6 payement Function Description 17
3.7 Generate feedback Function Description 17
3.8 Delete account Function Description 18

4.1 User 32
4.2 Admin 33
4.3 Presentation 33
4.4 AIProcessor 33
4.5 FeedbackReport 33
4.6 DatabaseManager 33
4.7 (SAVEE) Dataset Description 53
4.8 (TESS) Dataset Description 53
4.9 (RAVDESS) Dataset Description 54
4.10 (CREMA-D Dataset Description) 54
4.11 FER2013 Dataset Description 54
4.12 AffectNet Dataset Description 54
4.13 PodcastFillers Dataset Description 55
4.14 Test Cases for Scenario X 60
4.15 Test Cases for Scenario Y 64

List of Figures

1.1 Overview Diagram 4
1.2 Time plan table 6
1.3 Time plan chart 6

2.1 Orai 10
2.2 Speeko 11
2.3 Virtual Speech 11

3.1 Use Case 14

4.1 Context Diagram 30
4.2 Architectural Diagram 31
4.3 UML Diagram 32
4.4 UML Package Diagram 34
4.5 Pace Code 35
4.6 Grammar Code 36
4.7 Posture Code 37
4.8 Gesture Code 38
4.9 DeepFace Emotion Recognition Model 39
4.10 DeepFace Emotion Recognition Model 40
4.11 Tone code 41
4.12 Pronunciation code 42
4.13 Filler word model fine-tunning 43
4.14 Filler Word code 44
4.15 Sequence Diagram 46
4.16 User Sequence Diagram 47

4.17 AI Sequence Diagram 47
4.18 Admin Sequence Diagram 48
4.19 FER dataset sample 51
4.20 AffectNet dataset sample 52
4.21 NOSQL Schema Diagram 55
4.22 Sign Up/Login 56
4.23 Sign Up 56
4.24 Login 57
4.25 Signed Out 57
4.26 Upload 57
4.27 Settings 57
4.28 Profile 58
4.29 Edit 58
4.30 Admin Dashboard 58
4.31 Admin Dashboard 58

A.1 Git Insights 75
A.2 Git Insights 75
A.3 Git Insights 76
A.4 Git Insights 76
A.5 Git Insights 76

Chapter 1

Introduction

Effective presentation skills are crucial for success in both academic and professional settings. Therefore, we developed an AI-driven app to help students and professionals enhance their communication abilities. Not only does a strong presentation require well-structured content and clear speech, but it also depends on nonverbal cues such as body language, facial expressions, and vocal tone. However, refining these elements can be challenging without proper guidance. For this reason, our app allows users to upload presentation videos, which are then analyzed to provide detailed feedback on key aspects. As a result, this tool offers personalized insights, enabling users to communicate more confidently. Ultimately, it is invaluable for academic presentations, workplace proposals, and professional meetings.

1.1 Motivation

Presentations are a key part of sharing ideas, delivering research, and making an impact in academic and professional settings. Strong presentation abilities aid in capturing audiences' attention, establishing credibility, and effectively conveying messages in conferences, business meetings, and classrooms. Nonverbal cues like body language and facial expressions, which are essential for giving a confident and engaging presentation, are difficult for many people to master. Despite their significance, these abilities are frequently disregarded, depriving professionals and students of the right direction for growth. While professionals have to promote ideas, facilitate discussions, and influence decisions, students must clearly present their work in academic settings. Many people lose out on chances to improve their abilities and communicate clearly when they don't receive organized feedback.

1.1.1 Problem Statement

A major challenge in mastering public speaking is the lack of objective feedback on nonverbal communication. Without proper assessment, individuals struggle with facial expressions, body language, and vocal tone, which are crucial for engaging with audience. Limited access to professional coaching further hinders skill development, while difficulties in word choice and articulation make it harder to communicate ideas effectively. Without structured guidance, many find it difficult to deliver confident and impactful presentations.

1.2 Aims and Objectives

- Enhance Public Speaking Skills: Improve overall speaking abilities, focusing on delivery and body language.
- Comprehensive Analysis: Evaluate key presentation elements like speech clarity, facial expressions, and audience interaction.
- Personalized Feedback: Delivering tailored insights to help users recognize and improve specific areas of their performance.
- Progress Tracking: Enable users to monitor their development through progress reports.
- User-Friendly Experience: Develop an intuitive mobile app that simplifies recording and reviewing presentations.
- Interactive Learning: Facilitate practice with real-time audience interactions to simulate real-world scenarios.

1.3 Scope

The System will Contain:

- Uploading Functionality: Users can upload presentations via the mobile app.
- Speech Analysis: Analyzes speech clarity, pacing, tone, and filler words using NLP.
- Body Language Assessment: Evaluates gestures and facial expressions through computer vision.
- Audience Interaction Evaluation: Assesses responses to audience questions.
- Personalized Feedback: Generates tailored feedback reports.
- Progress Tracking: Allows users to monitor improvement with visual reports.

1.4 System Overview

This figure 1.1 illustrates the sequence of operations in the system, which follows a structured pro- cess of input, pre-processing, analysis, and feedback generation. The system begins with the user uploading a video of their presentation, which undergoes preprocessing to extract key fea- tures from speech, facial expressions, gestures, and movement for accurate evaluation. In the analysis phase, speech processing uses NLP to assess clarity, pacing, grammar, and filler words, while response quality is evaluated to determine how effectively the speaker answers questions. Audience interaction analysis measures confidence and engagement, particularly during inter- actions with listeners. The system also applies emotion detection to identify ner- vousness, en- thusiasm, and other emotional states that impact delivery. For body language assessment, the system examines facial expressions, posture, gestures, and movement, ensuring that the speaker maintains a professional and engaging presence. The system also ensures that movement re- mains natural and not distracting throughout the presentation. Once all analyses are complete, AI models generate personalized feedback, highlighting strengths, mistakes, and areas for im- provement. The system provides a score out of 10 along with a star rating system, helping users track progress and refine their presentation skills effectively .

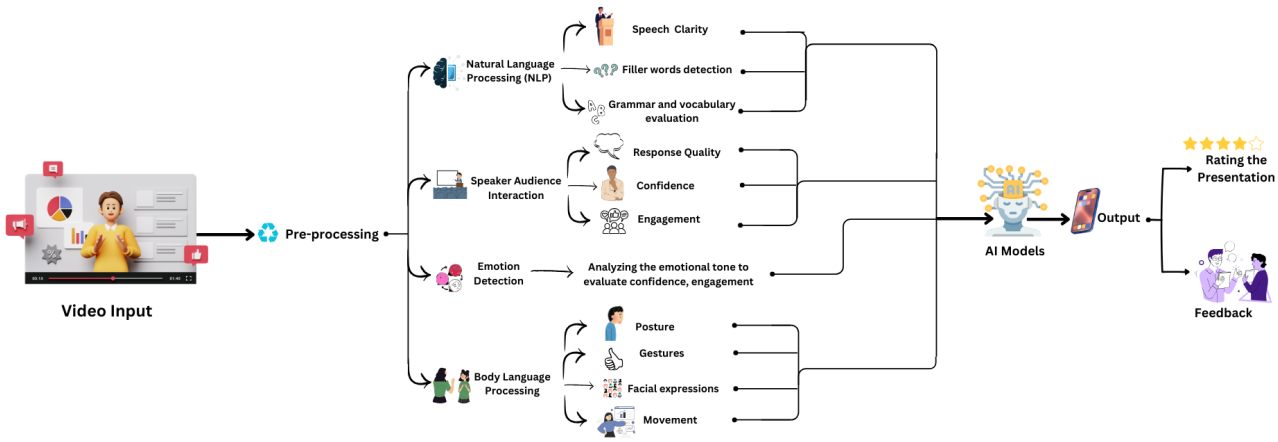

Figure 1.1: Overview Diagram

1.4.1 Business Context

Our AI-powered application helps professionals, students, and businesses improve their presentation skills by analyzing speech clarity, tone, body language, and facial expressions. It provides personalized feedback to enhance public speaking effectiveness and confidence. Users can upload videos to receive detailed performance reports and track their progress over time. Designed for corporate training, education, and professional development, it supports individuals in refining their communication skills. Our application ensures scalable and accessible training for businesses and academic institutions.

1.4.2 Users Characteristics

Our application is accessible, practical, and beneficial for both individuals and institutions, helping users improve their presentation skills in academic and professional settings.

- The application is easy to use and suitable for both beginners and experienced speakers.
- Users should know how to record and upload videos to receive AI-driven feedback.
- It analyzes speech clarity, body language, and facial expressions to provide personalized insights.
- Students, professionals, public speakers, educators, and businesses can use it to enhance their communication skills..
- Educators, trainers, and businesses can integrate it into training programs to support skill development.

1.5 Project Management and Deliverables

1.5.1 Time Plan

Task	Start Date	End Date	Duration (days)
Idea and Supervisors	12.7.24	9.9.24	52
Information collection and researches	15.9.24	5.10.24	20
Survey and Proposal Preparation	6.10.24	19.10.24	13
Proposal Evaluation 10%	20.10.24	27.10.24	7
SRS Preparation	1.11.24	21.12.24	50
SDD Preparation	1.1.25	7.8.25	37
SRS/SDD Evaluation 65%	8.2.25	21.2.25	13
Technical Working Implementation	1.3.25	14.4.25	55
Technical Evaluation 80%	14.4.25	21.4.25	7
Test and Validate	22.4.25	7.5.25	15
Final Prototype 90%	7.5.25	15.5.25	7
Final Thesis 100%	21.6.25	29.6.25	8

Figure 1.2: Time plan table

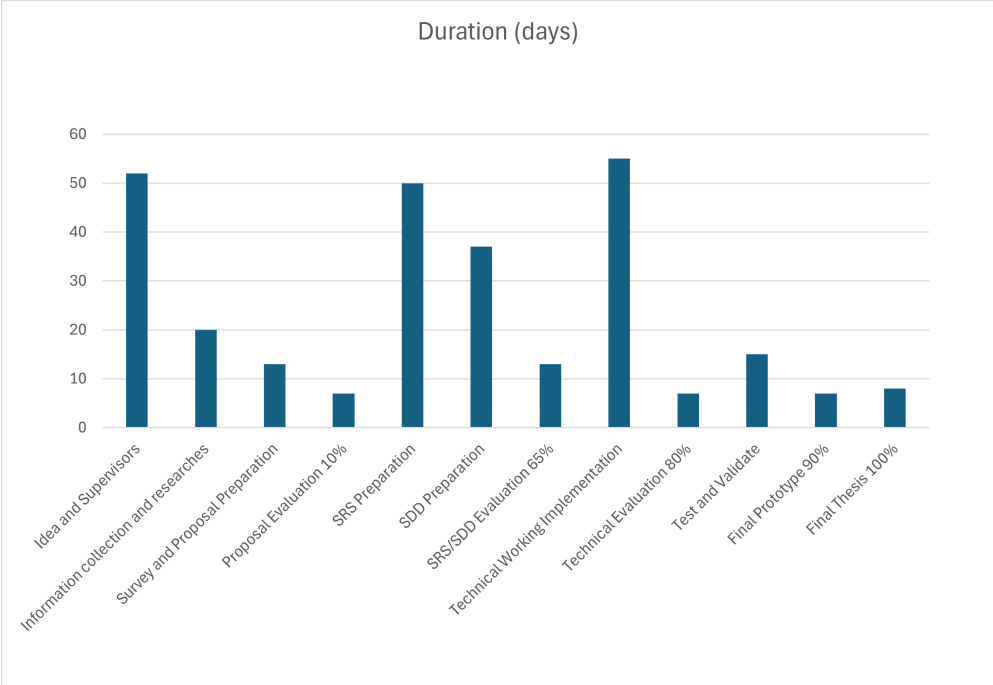

Figure 1.3: Time plan chart

1.5.2 Deliverables

- The project will produce a mobile application that give the user a feedback on their presentations focusing on body language, facial expressions, eye contact, speech fluency, grammatical correctness, and overall appearance.Also a various number of papers on topics like AI feedback systems, non-verbal behavioral analysis and NLP based speech analysis will be included,
- **deliverables are**:
  - Proposal document
  - Academic Papers
  - SRS document
  - SDD document
  - Final Thesis document
  - Mobile Application

Chapter 2

Background and Related Work

2.1 Introduction

This chapter provides an overview of the background and related work in AI-driven communication and training systems. It first explores the role of AI in improving public speaking and presentation skills, emphasizing its impact on language learning, speech analysis, and automated performance evaluation. It then examines challenges in verbal and nonverbal communication, highlighting the need for AI-driven solutions. The chapter also reviews similar systems in academic and business applications, analyzing their methodologies, strengths, and limitations. Finally, a summary of key findings is presented, outlining how our application addresses existing gaps and improves presentation training.

2.2 Background

This project falls under the field of Artificial Intelligence in Communication and Training Systems. AI has significantly transformed multiple industries, including education, professional development, and corporate training, by enabling personalized learning experiences. In particular, AI-driven solutions have been widely adopted in language learning, speech analysis, and automated performance evaluation. Our application leverages these advancements to enhance public speaking and presentation skills by providing automated feedback on speech clarity, body language, and facial expressions. Many individuals struggle with effective communication, and

traditional training methods often lack objectivity, accessibility, and scalability. Furthermore, identifying issues in nonverbal communication—such as facial expressions, hand gestures, and posture—can be challenging without expert guidance. By integrating Natural Language Processing (NLP), Computer Vision, and Deep Learning, our application evaluates both verbal and nonverbal communication in presentations. Users can upload videos, receive detailed AI-generated feedback, and track their progress over time. This ensures an efficient, scalable, and data-driven solution to help individuals refine their presentation skills and communicate with confidence.

2.3 Related Systems

2.3.1 Academic

- **Almubarak, Alhalabi, Albidewi & Alharbi** mentioned in the paper [1] the creation of teacher evaluation system based on T-KPIs and Vision 2030, using two algorithms: one for overall teacher performance and another for specific elements. AI techniques provide data-driven analysis, with evaluative weights determined through expert workshops. Performance elements are assigned percentage weights, such as 100% for "Job Performance," and 6% for "teacher commitment to standard language." Expert input ensures a balanced, adaptable evaluation process that fits various educational settings.

- **Zou, Du, Wang, Chen & Zhang** in this paper [30] looked into how well AI speech evaluation tools can help EFL students become more proficient speakers. Significant improvements were noted by the participants; 80% noted gains in pronunciation, 75% in fluency, and 87.5% in reading aloud. 32.5% of users engaged with the programs daily, and 37.5% did so several times a week, indicating frequent usage. Popular applications like Liulishuo, IELTS Liulishuo, EAP Talk, and Shanbay were among the AI models utilized. Concerns regarding the accuracy of the feedback, however, were voiced, suggesting a discrepancy between the outcomes and expectations. To improve user experience, the study suggested tracking progress and increasing the accuracy of feedback. Although the

programs were successful overall, resolving feedback issues is essential to maximizing their advantages.

- **Younis, Mohsen, Houssein & Ibrahim** in [29] provide an extensive overview of various machine learning (ML) and deep learning (DL) algorithms employed in the field of emotion recognition, along with their reported accuracies. Notable algorithms include Naive Bayes (NB), which achieved an F1 score of over 90% in binary classification and an average accuracy of 74%; Support Vector Machine (SVM), with an average accuracy of 85.47% and a maximum of 78.97% in another study; and Random Forest (RF), which reported an impressive accuracy of 95.6%. Convolutional Neural Networks (CNN) reached an accuracy of 80%, while in a hybrid model, it achieved an F1 score of 80.76%. Decision Tree (DT) had an average accuracy of 74%. Among deep learning techniques, the Bidirectional Gated Recurrent Unit (Bi-GRU) achieved an accuracy of 79.46%, and a hybrid model combining CNN, Bi-GRU, and SVM yielded an F1 score of 81.27% and an accuracy of 80.11%. Additionally, ensemble learning methods demonstrated significant potential, with the best accuracy reaching 98.2% through stacking techniques. The review highlights the advancements in emotion recognition technologies and their promising applications across various fields.

2.3.2 Business Applications

- **Orai** is an AI-driven voice coaching app designed to enhance public speaking skills by providing instant feedback. Users can practice their speeches, and the app analyzes various aspects such as tone, pitch, volume, clarity, and the use of filler words.

Figure 2.1: Orai

- **Speeko** focuses on helping users refine their public speaking skills through voice recognition technology. The app provides feedback on pacing, intonation, and articulation while offering a range of challenges to practice different speaking scenarios.

Figure 2.2: Speeko

- **Virtual Speech** combines online learning with virtual reality (VR) training environments to simulate real-world speaking scenarios. Users can practice in various settings with virtual audiences, which helps reduce anxiety and improve delivery.

Figure 2.3: Virtual Speech

2.4 Summary

This chapter explores the background and related work in AI-driven communication and training systems. It discusses how AI has improved language learning, speech analysis, and automated performance evaluation, highlighting the need for objective and scalable public speaking training. The chapter then reviews similar systems in academic and business applications, analyzing their features and limitations. In the academic field, various AI tools assess teacher performance, speech evaluation, and emotion recognition, while business applications like Orai, Speeko, and Virtual Speech provide AI-based feedback for public speaking. The chapter concludes by summarizing key findings and emphasizing how our application addresses gaps in existing solutions.

Chapter 3

System Requirements Specification

3.1 Introduction

This chapter defines the functional and non-functional requirements of the AI-powered presentation analysis system. It outlines the essential features, constraints, and expectations that ensure effective performance.This system is designed to help users improve their presentation skills by analyzing body language, facial expressions, voice tone, and speech. Unlike existing solutions that focus solely on speech transcription or basic sentiment analysis, it provides a comprehensive evaluation with targeted feedback, allowing users to refine their delivery, clarity, and engagement.The following sections detail the technical and user requirements necessary for successful implementation, ensuring the system meets its objectives while maintaining usability, efficiency, and scalability.

3.2 Functional Requirements

3.2.1 System Functions

ID:1 User should sign up for the mobile application by providing their credentials.

ID:2 User and Admin can log in and log out.

ID:3 System validates email .

ID:4 system validates password

ID:5 System should measure word pacing.

ID:6 System should analyze grammar of the user.

ID:7 System should analyze prounancation of the user.

ID:8 System should analyze filler words of the user.

ID:9 System should detect pitch,energy and emotion from tone .

ID:10 System should analyze body language, including posture.

ID:11 System should analyze Hand gestures.

ID:12 User can upload a video of their presentation for analysis.

ID:13 User can receive detailed feedback on their presentation performance.

ID:14 User can track their progress over time through visual reports.

ID:15 User can request support from the admin .

ID:16 User can delete their account permanently, including all associated data .

ID:17 Admin can edit user profiles.

ID:18 Admin can assist users in User Support by addressing technical issues.

ID:19 Admin can view user feedback provided by the system.

ID:20 Admin can manage user submissions by reviewing uploaded presentations.

ID:21 Admin can monitor user progress tracking through detailed performance reports.

Figure 3.1: Use Case

3.2.2 Detailed Functional Specification

Table 3.1: SingUp Function Description

Name	Sign Up
Code	FR01
Priority	high
Critical	This Function is essential because it enables users to register for the system—without which they are unable to access any features
Description	This Function allows users to register for the system by providing their emails and passwords
Input	User details (Full Name,Email,Password)
Output	Boolean (true for successful registration, false for failure)
Pre-condition	The user must be on the Sign Up screen and fill out the required fields with correct data.
Post-condition	User is redirected to the home page after a success registration, and their account is provided in the system.
Dependency	This function checks the correctness of user-provided data by focusing on the input validation function For the purpose to save user data, it could also depend on the database function.
Risk	inaccurate input data-server failures-database problems

Table 3.2: Log In Function Description

Name	Log In
Code	FR02
Priority	High
Critical	Enables existing users to access their accounts.
Description	Authenticates users by verifying their email and password.
Input	User credentials: email and password
Output	Boolean (true for success, false otherwise)
Pre-condition	User must have an existing account, and the database must be operational.
Post-condition	After successfully logging in, the user is given access to the system and taken to the homepage.
Dependency	Requires Sign Up functionality to create user accounts and database connectivity for credential verification.
Risk	Risks include technical problems that might stop users from logging in or incorrect login credentials that could prevent access.

Table 3.3: Sign Out Function Description

Name	Sign Out
Code	FR03
Priority	high
Critical	This Function is essential because it allows users to log out of the system, ensuring secure sessions and preventing unauthorized access.
Description	This Function allows users to log out of the system, terminating their current session.
Input	User request to log out (clicking the sign-out button).
Output	Boolean (true for successful logout, false for failure).
Pre-condition	The user must be logged into the system.
Post-condition	User is redirected to the login page, and the session is terminated.
Dependency	Depends on the session management function, which tracks user login status.
Risk	Server failures, session management errors.

Table 3.4: Edit Profile Function Description

Name	Edit Profile
Code	FR05
Priority	High
Critical	Core functionality for user profile management.
Description	Allows users to edit their own profile information. Changes can include name, email, password, and other personal details.
Input	User profile data (name, email, password)
Output	Boolean (true for success, false otherwise). Updated user profile information.
Pre-condition	User must be logged in.
Post-condition	User's profile is updated, and changes are reflected in the system.
Dependency	Depends on user authentication and authorization (admin privileges for admins).
Risk	Risk of incorrect data input or unauthorized changes. Action: Implement proper validation and access control to prevent unauthorized access and updates.

Table 3.5: Upload Video Function Description

Name	Upload Video
Code	FR04
Priority	High
Critical	Core functionality required for video analysis.
Description	Allows users to upload a video file for processing and analysis. Upon upload, the AI automatically begins analyzing speech, body language, facial expressions, voice tone, and other presentation elements.
Input	Video file (.mp4, .avi, .mov)
Output	Boolean (true for success, false otherwise) and immediate AI analysis initiation on speech, body language, facial expressions, tone, and other presentation aspects.
Pre-condition	User must be logged in, and the video must meet size and format requirements.
Post-condition	Video is stored on the server, and AI analysis starts immediately on speech, body language, facial expressions, tone, and other presentation elements.
Dependency	Depends on the Sign Up and Log In functionality for user authentication. Requires a valid server connection for video upload.
Risk	Risk of upload failure due to unsupported file formats or large sizes. Action: Implement file validation and size checks before upload.

Table 3.6: payement Function Description

Name	Payment Packages
Code	FR06
Priority	High
Critical	Core functionality for payment processing and package selection.
Description	Allows users to choose from multiple payment packages (Basic, Premium, Enterprise). Each package offers different features and benefits. Users can select and make payments for their preferred package.
Input	Package selection (Basic, Premium, Enterprise)
Output	Boolean (true for successful payment and package selection, false otherwise). Confirmation of selected package and payment details.
Pre-condition	User must be logged in and have access to the available payment packages. User's payment method must be valid.
Post-condition	User's package selection is processed, and payment is completed successfully. The selected package is applied to the user's account.
Dependency	Depends on the user authentication system and a valid payment gateway for processing payments.
Risk	Risk of payment failure due to invalid payment methods or technical issues with the payment gateway. Action: Implement error handling and validation for payment methods.

Table 3.7: Generate feedback Function Description

Name	Generate Feedback (Post-Upload)
Code	FR09
Priority	High
Critical	Provides essential feedback to the user after video upload for it's performance.
Description	After the user uploads their video, the system generates and displays feedback based on AI analysis of the presentation, covering aspects such as speech, body language, tone, and overall performance. The feedback is presented in a popup, offering insights and suggestions for improvement.
Input	Video file (uploaded by user)
Output	Feedback (textual feedback based on AI analysis, displayed as a popup message)
Pre-condition	User must have successfully uploaded their video. Video must meet format and size requirements. AI analysis should be completed before feedback generation.
Post-condition	A popup message is displayed to the user containing actionable feedback on their video.
Dependency	Requires successful video upload and completion of AI analysis (speech, body language, tone).
Risk	Risk of delayed or inaccurate feedback generation if AI analysis takes longer or fails. Action: Ensure that the AI analysis completes successfully before triggering feedback and provide user-friendly error handling if issues arise.

Table 3.8: Delete account Function Description

Name	Delete Account
Code	FR07
Priority	High
Critical	Core functionality for user account management.
Description	Allows users to delete their own account. Deleting an account removes all associated data and access to the system.
Input	User ID (for admin), user confirmation (e.g., "Are you sure you want to delete your account?")
Output	Boolean (true for success, false otherwise). Confirmation of account deletion.
Pre-condition	User must be logged in. User must confirm the account deletion action.
Post-condition	User account is permanently deleted from the system..
Dependency	Depends on user authentication and authorization (admin privileges for admins).
Risk	Risk of accidental account deletion. Action: Implement double confirmation for deletion and proper user feedback.

3.3 Interface Requirements

3.3.1 User Interfaces

The user interface is the main way users will interact with the system, so it should be simple, intuitive, and easy to navigate. It will allow users to upload presentation videos and view detailed feedback on body language, facial expressions, tone, and speech quality. Developed using Flutter, the interface will offer a smooth, cross-platform experience with clear, interactive reports.

3.3.2 Graphical User Interface (GUI)

The Graphical User Interface (GUI) of the application is designed to provide a user-friendly and intuitive experience. It facilitates seamless navigation, easy video uploads, and clear feedback visualization for enhancing presentation skills.

- **Opening Screens :** Serve as the initial landing screens when the app is launched, offering a welcoming interface that introduces the app's features and guides users to begin their journey.
- **Sign-Up Screen:** Enables new users to create an account by entering personal details such as name, email, and password. It includes form validation to ensure secure and accurate data entry.
- **Login Screen:** Allows existing users to securely log into the application using their credentials. It includes options for password recovery.
- **Home Screen:** Provides an overview of the application with quick access to key features such as uploading presentations, viewing feedback, and tracking progress.
- **Profile Screen:** Displays user-specific information such as name, email, and account details. Users can view their activity and personal information in one place.
- **Edit Profile Screen:** Provides functionality for users to update their personal details, including profile picture, contact information, and password.

- **Settings Screen:** Offers options for users to customize their app experience, manage notifications, privacy settings, and account preferences.
- **Video Upload Screen:** Allows users to upload pre-recorded presentation videos. It includes drag-and-drop functionality, file selection options, and progress indicators for upload status.
- **Feedback Display Screen:** Presents detailed feedback on the uploaded presentation, including analysis of body language, speech clarity, tone, and overall performance. Feedback is visualized through charts, graphs, and score summaries.
- **Progress Tracking Screen:** Enables users to monitor their improvement over time through graphical representations of their performance metrics. This includes trend analysis and historical data comparisons.
- **Support Access Screen:** Provides users with access to support resources, including FAQs, troubleshooting guides, and direct communication with the support team for assistance with technical issues or general inquiries.
- **About App Screen:** Provides information about the application, including its purpose, key features, version details, and developer contact information.
- **Logout Screen:** Allows users to securely log out of their accounts, ensuring their session data is cleared and account security is maintained.
- **Admin Dashboard:** Offers administrative tools for managing user accounts, monitoring system performance, reviewing user submissions, handling user support requests, and overseeing system analytics. This section is accessible only to authorized administrators and developers to ensure system security.
- **Navigation Menu:** Provides easy access to different sections of the app, such as Home, Upload, Feedback, Progress, Profile, and Settings. The menu is designed for quick transitions and an intuitive user experience.

- **Interactive Elements:** Includes buttons (e.g., Upload, Submit, Download Feedback), icons (e.g., Profile, Notifications), input fields for user information.
- **Responsive Design:** The interface is optimized for various screen sizes, ensuring a consistent experience on both mobile devices and desktops.

3.3.3 Hardware Interfaces

The system is designed to work seamlessly on standard mobile and desktop hardware, requiring only basic functionalities for tasks like file uploads. It does not need any specialized hardware interfaces or advanced device capabilities. Users can upload pre-recorded presentation videos using their existing devices without additional hardware support.

3.3.4 Communication Interfaces

The communication interfaces primarily involve network connections that facilitate data transfer between the mobile application (client-side) and the backend server (server-side processing). The mobile application communicates with the server over a secure internet connection using HTTP/HTTPS protocols.

- **Secure Data Transmission:**
  Video files uploaded from the mobile app are securely transmitted to the backend server for AI-based analysis using HTTPS with TLS/SSL encryption, ensuring data confidentiality and integrity.
- **Data Exchange:**
  After processing, the server sends the analysis results (feedback on body language, tone, speech clarity, etc.) back to the mobile application for the user to review.
- **API Communication:**
  The communication between the client and server is facilitated through RESTful APIs, enabling efficient and structured data exchange.

- **Error Handling & Data Integrity:**
  The system includes mechanisms to detect data transfer errors and ensure the integrity of files during upload and download processes.

- **Network Requirements:**
  A stable internet connection is required to ensure smooth and efficient data transmission between the client and server.

3.3.5 Application Programming Interface (API)

The system will use Application Programming Interface (API) facilitates communication between the mobile application and the backend server, enabling the mobile app to send data for processing and retrieve results. Below are the key functions of the API:

- **Video Upload Function:** This function allows the mobile app to send a video file to the backend server for processing. Once the video is uploaded, the server processes it and provides feedback to the app.

- **Result Retrieval Function:** This function allows the mobile app to retrieve the analysis results after the video is processed. The feedback includes information on various aspects such as body language, speech tone, grammar, and overall presentation skills.

- **Supported Video Formats Check:** This function ensures that the mobile app can verify the supported video formats before the user attempts to upload a video. It checks whether the uploaded video is compatible with the backend system for successful processing.

3.3.6 External APIs

The system will integrate the following external APIs to enhance performance, security, and user engagement.

- **Cloud Storage APIs:** Securely store and manage large video files to ensure efficient data handling and accessibility.

- **Authentication & Security APIs:** Manage user registration, login processes, and ensure robust data security through secure authentication mechanisms.

- **Video Streaming APIs:** Optimize video playback within the application, providing smooth streaming experiences for presentation reviews.

- **Analytics APIs:** Track user behavior and engagement metrics to support continuous app improvement and data-driven decisions.

- **Payment Processing APIs :** Facilitate secure online payment transactions, enabling subscription models or premium feature access.

External Libraries

Illustrate essential functionalities that enhance system performance by enabling efficient processing, seamless integration, and optimized resource utilization.

Computer Vision & Body Language Analysis :

- **OpenCV (cv2)** – Image and video processing, including frame extraction.
- **MediaPipe (mediapipe)** – Body posture and gesture recognition.

Audio & Speech Processing :

- **SpeechRecognition (speech_recognition)** – Converts speech to text.
- **Pydub (pydub.AudioSegment)** – Audio format conversion and processing.
- **Whisper (whisper)** – AI-based speech-to-text conversion.
- **Librosa (librosa)** – Extracts features from audio for analysis.
- **MoviePy (moviepy.editor)** – Processes and extracts audio from video files.

File Handling & System Utilities :

- **OS (os)** – File and system operations.
- **Shutil (shutil)** – File management utilities.
- **AIFC (aifc)** – Handling AIFF and AIFC audio formats.

Natural Language Processing (NLP) :

- **Transformers (T5ForConditionalGeneration, T5Tokenizer)** – Text summarization and grammar correction.
- **SpaCy (spacy)** – Grammatical analysis and NLP tasks.
- **Difflib (difflib)** – String comparison for text analysis.

Deep Learning:

- **TensorFlow (tensorflow)** – An open-source deep learning framework used for building and training neural networks, especially for image and video data processing.
- **Keras (keras)** – A high-level neural networks API running on top of TensorFlow, used to simplify the creation and training of deep learning models.
- **Torch (torch)** – A flexible deep learning framework often used for advanced model development and optimization.
- **Transformers (AutoModelForAudioClassification)** – A library designed for handling state-of-the-art models in NLP and audio classification tasks.

Data Handling & Preprocessing :

- **Pandas (pandas)** – Data processing and analysis.
- **NumPy (numpy)** – Numerical computations.
- **Scikit-learn (sklearn)** – Data splitting for deep learning models

3.4 Design Constraints

3.4.1 Standards Compliance

The application will be available for both Android and iOS devices and can be downloaded from the Google Play Store and Apple App Store.A smartphone with an active internet connection is required for uploading and analyzing presentations.

3.4.2 Software Constraints

- **Programming Frameworks :** The mobile application will be developed using Flutter, ensuring cross-platform compatibility for Android and iOS.
- **AI Models :** The system will utilize NLP models and Deep Learning techniques, including CNNs for visual analysis and RNNs for speech processing.
- **Libraries & Tools :**
  - **Speech Analysis :** SpeechRecognition for speech-to-text conversion, pydub for handling audio files, and T5-Base-Grammar-Correction for grammar analysis.
  - **Body Language Analysis :** MediaPipe will handle body pose estimation and hand tracking, while OpenPose will support full-body movement tracking.
  - **Facial Expression Analysis :** TensorFlow and Keras for model training, Dlib for advanced face recognition, and DeepFace for emotion detection.
- **Database & Storage :** A secure database solution will store user data and AI-generated feedback while ensuring data encryption and privacy compliance.
- **Cloud & API Integration :** The system will support real-time AI processing through cloud-based APIs to ensure efficient presentation evaluation.

3.5 Non-functional Requirements

3.5.1 Security

- Ensure data confidentiality by encrypting stored and transmitted data.
- All user passwords must be hashed or encrypted to prevent unauthorized access.
- Implement user authentication and role-based access control to restrict data access.
- Preserve data integrity to prevent unauthorized modifications.
- Restrict access to protected mobile pages for unauthorized users.

3.5.2 Availability

- The system must maintain 99% uptime, ensuring continuous access for users.
- Automated backups must be implemented to prevent data loss.
- The application must run on a stable API to minimize downtime and ensure smooth functionality.

3.5.3 Maintainability

- The system must follow a modular architecture, making updates and expansions easier.
- Regular performance monitoring and error logging must be conducted .

3.5.4 Portability

- The system must be compatible with multiple platforms, including Android,Ios.
- The application should adapt to different screen sizes and resolutions for consistent usability.

3.5.5 Usability

- The system must be user-friendly, intuitive, and easy to navigate.
- It should provide multilingual support (English& Arabic) for accessibility.
- It ensures the interface is simple and easy to understand
- Enable efficient communication between patients and doctors

3.5.6 Accessibility

- The system must be fully responsive, ensuring seamless use across devices.
- Mobile pages must be keyboard-friendly and easy to navigate for accessibility.

3.5.7 Extensibility

- Design the system to be scalable, allowing for future feature enhancements.
- Support integration with third-party applications, such as external AI models.
- Ensure the architecture supports new AI models and data sources without major restructuring.

3.6 Summary

This chapter outlines the system requirements for the project, detailing both functional and non-functional aspects. It introduces the system's purpose and highlights the improvements it offers over existing solutions. The functional requirements section describes the system's core functionalities, specifying use cases and detailed functional specifications. Interface requirements cover how the system interacts with users through graphical and command-line interfaces, as well as hardware and communication interfaces. Additionally, the chapter provides information on APIs and external libraries used in the system. Design constraints, including software, hardware, and standards compliance, are also addressed. Finally, non-functional requirements such as security, reliability, and scalability are discussed to ensure the system meets performance expectations.

Chapter 4

System Design

4.1 Introduction

This chapter outlines the system design of the AI-powered Presentation Skills Evaluation Application, focusing on its architecture and key components. It details how the system's design ensures automation, scalability, and accessibility, enabling users to receive AI-driven analysis and personalized feedback on their presentation skills. By leveraging natural language processing, deep learning, and computer vision, the system provides a comprehensive and efficient solution for improving speech clarity, body language, and overall delivery.

4.2 Architecture Design viewpoints

4.2.1 Context viewpoint

Actors and Their Roles:

- **Users:** Upload and store presentation videos for evaluation and Receive personalized feedback to improve presentation skills.
- **Mobile Application:** Manages user interactions, video uploads, and feedback delivery.
- **AI Processing Engine:** Analyzes facial expressions, body language, tone, and speech clarity and Generates a comprehensive feedback report.

Systems services:

- **Video Upload & Storage:** Users upload their presentation videos, which are securely stored in the database.
- **AI-Powered Analysis:** The system processes videos using AI to assess key presentation elements.
- **Feedback Generation:** A detailed performance report is generated, highlighting strengths and areas for improvement.

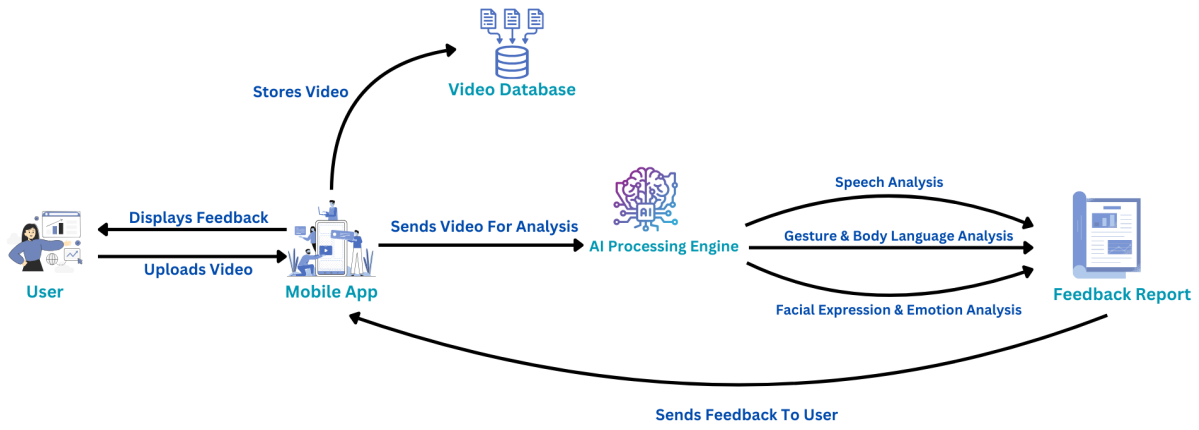

Figure 4.1: Context Diagram

4.2.2 Composition viewpoint

In Figure 4.2

- **Presentation Layer** (User & Admin Interface): Users upload videos and receive feedback, Admins manage settings.
- **Application Layer** (Processing Controllers): Handles video, audio, and gesture analysis before sending data to AI.
- **AI Model Layer** (Deep Learning & AI Processing): Uses NLP, Computer Vision, and Deep Learning to analyze speech and gestures, and generate feedback.
- **Data Access Layer** (Database & Storage): Stores videos, reports, and AI analysis results.

- **Infrastructure & Security Layer**: Manages authentication, exception handling, data protection and protects user data and enforce role-based access.

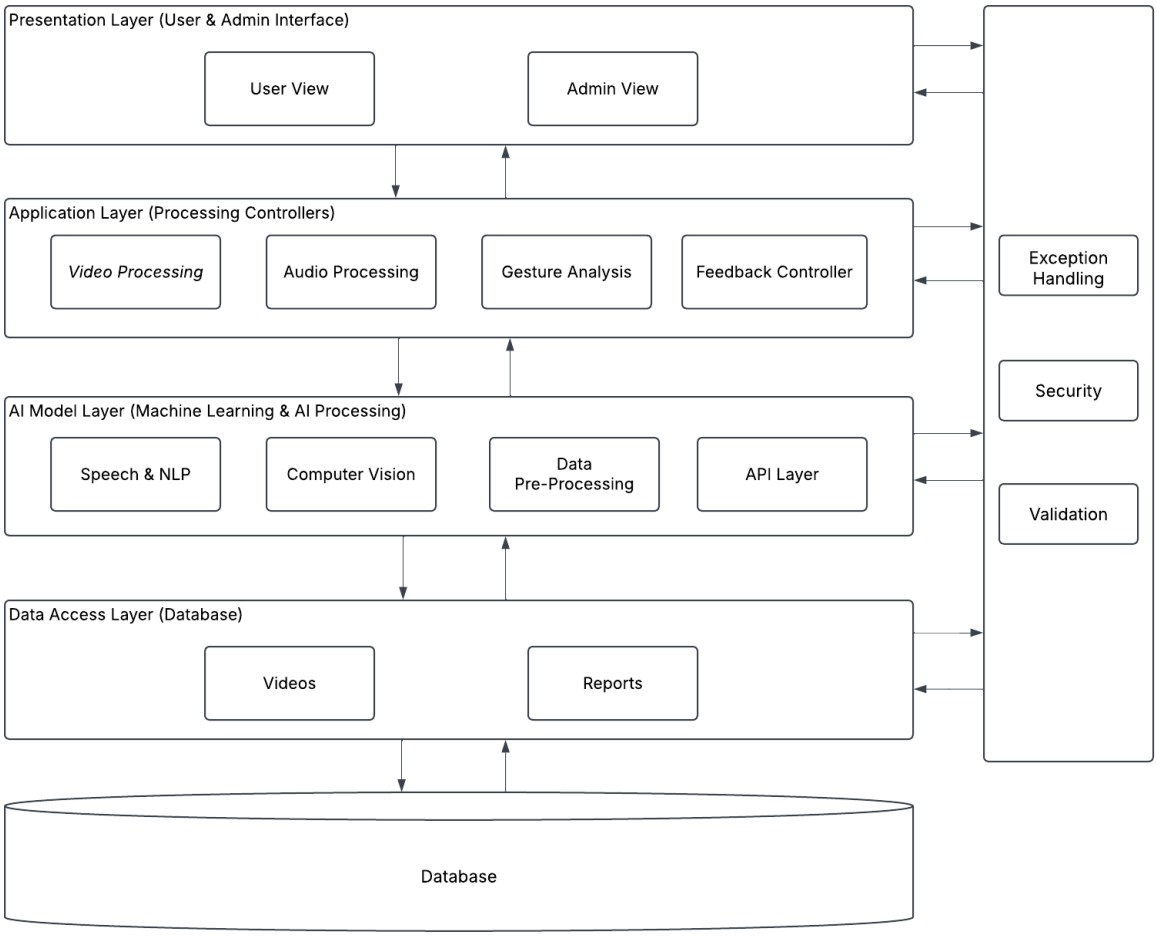

Figure 4.2: Architectural Diagram

Design Rationale

The System is built on a layered architecture to ensure scalability, modularity, and security. This structured approach separates system components into distinct layers, making it easier to develop, maintain, and expand without disrupting overall functionality. The modular design also allows for future upgrades, such as enhanced AI models, improved user experience, and optimized performance. It leverages AI-driven analysis, including Natural Language Processing, Computer Vision, and Deep Learning, to evaluate speech clarity, facial expressions, and body language. By analyzing these key presentation aspects, it generates automated, real-time feedback, enabling users to improve their public speaking skills and communication effectiveness.

4.2.3 Logical viewpoint

The UML Class Diagram 4.3 illustrates the main system components, their relationships, and how they interact. The User class represents standard users, while the Admin class extends User with additional management functions. The AIProcessor handles speech and gesture analysis, generating FeedbackReport objects, which are stored using DatabaseManager.

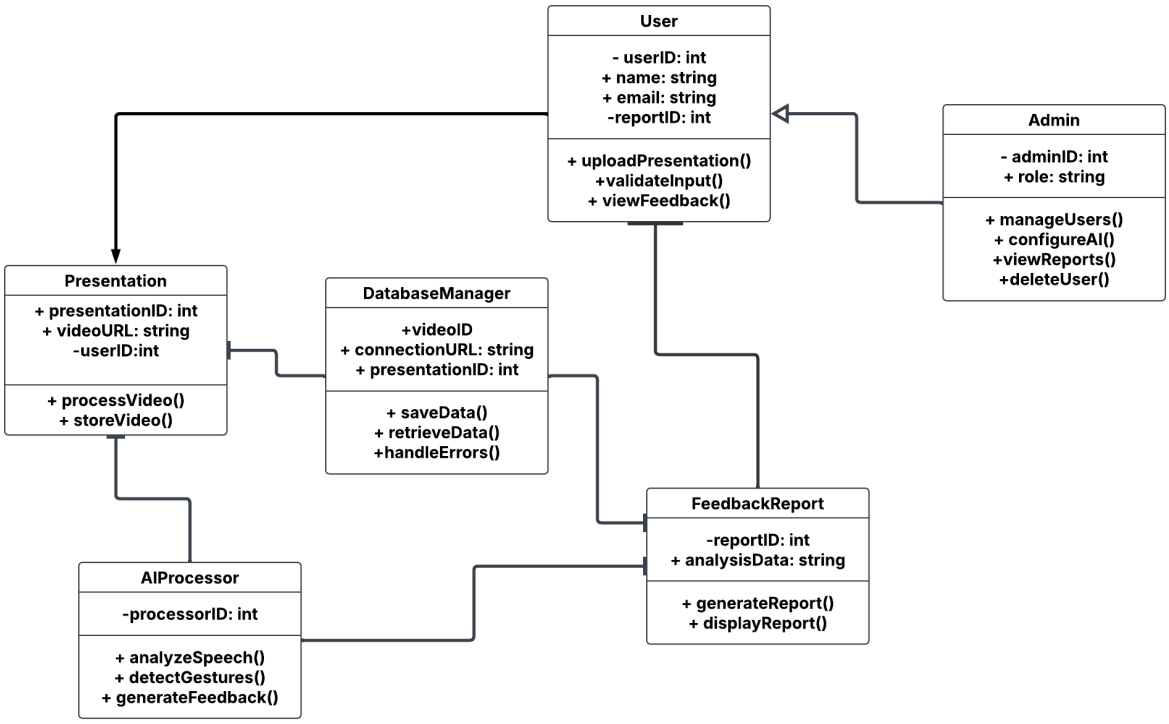

Figure 4.3: UML Diagram

Table 4.1: User

Abstract or Concrete:	Concrete
Superclasses	None
Subclasses	Admin
Purpose	Represents a system user
Collaborations	Works with Presentation, FeedbackReport
Attributes	userID: int, name: string, email: string
Operations	uploadPresentation(), viewFeedback(), ValidInput()

Table 4.2: Admin

Abstract or Concrete:	Concrete
Superclasses	User
Subclasses	None
Purpose	Manages users & AI settings
Collaborations	Manages AIProcessor, DatabaseManager
Attributes	adminID: int, role: string
Operations	manageUsers(), configureAI(), viewReports(),deleteUser()

Table 4.3: Presentation

Abstract or Concrete:	Concrete
Superclasses	None
Subclasses	None
Purpose	Stores uploaded videos
Collaborations	Used by AIProcessor
Attributes	presentationID: int, videoURL: string
Operations	processVideo(), storeVideo()

Table 4.4: AIProcessor

Abstract or Concrete:	Concrete
Superclasses	None
Subclasses	None
Purpose	Processes video for AI analysis
Collaborations	Works with Presentation & FeedbackReport
Attributes	processorID: int
Operations	analyzeSpeech(), detectGestures(),generateFeedback()

Table 4.5: FeedbackReport

Abstract or Concrete:	Concrete
Superclasses	None
Subclasses	None
Purpose	Stores AI-generated feedback
Collaborations	Generated by AIProcessor, viewed by User
Attributes	reportID: int, analysisData: string
Operations	generateReport(), displayReport()

Table 4.6: DatabaseManager

Abstract or Concrete:	Concrete
Superclasses	None
Subclasses	None
Purpose	Handles video & report storage
Collaborations	Works with Presentation & FeedbackReport
Attributes	databaseID: int, connectionURL: string
Operations	saveData(), retrieveData(),handleErrors()

4.2.4 Patterns use viewpoint

The MVC architecture ensures clear separation of concerns, while Singleton, Factory, and Observer patterns help in optimizing security, AI processing, and feedback handling. The diagrams 4.4 illustrate how these patterns are applied, providing a structured foundation for system design.

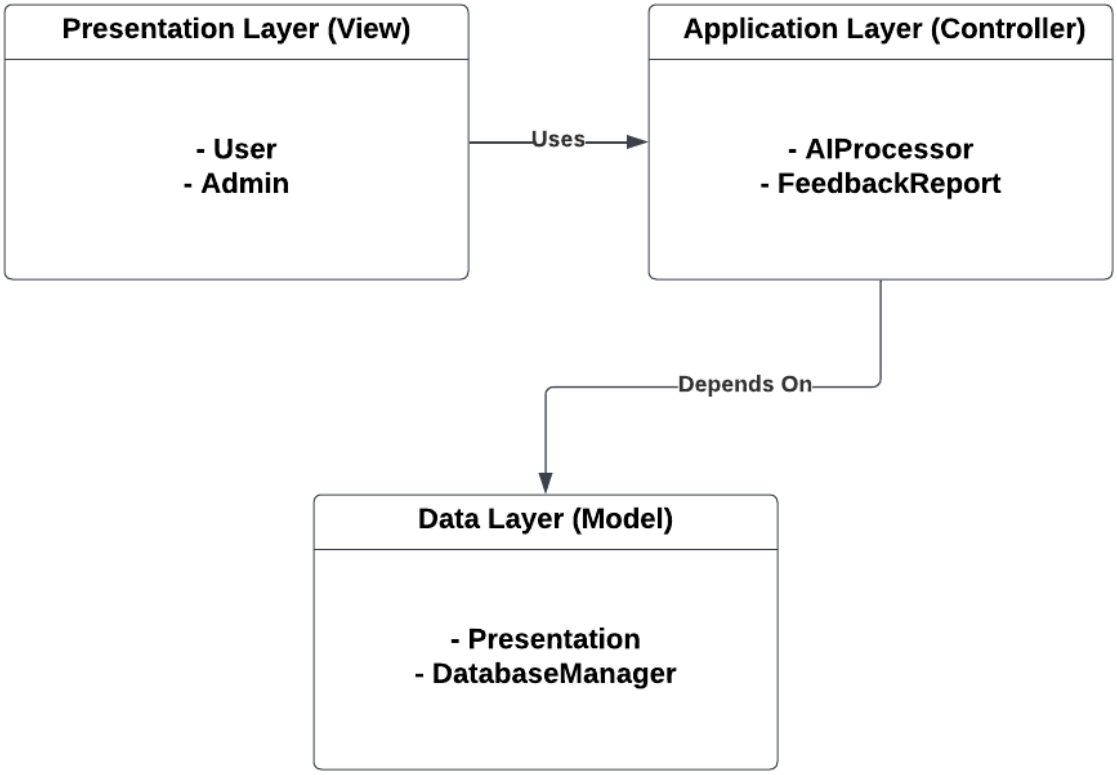

Figure 4.4: UML Package Diagram

Design Rationale

- MVC improves scalability and modularity.
- Singleton prevents multiple instances of DatabaseManager, avoiding conflicts.
- Factory Pattern allows AI modules to be extended easily.
- Observer keeps User updated when feedback is available.

4.2.5 Algorithm viewpoint

calculate_speech_pace Function 4.5 Determines the words per minute (WPM) from a given speech audio file and provides feedback.

- Load the audio file.
- Convert speech to text.
- Count words and calculate WPM.
- Categorize the pace into five levels (Slow, Fast, Perfect, etc.).

```python
def calculate_speech_pace(audio_file):
    audio_segment = AudioSegment.from_file(audio_file)
    # Get the duration of the audio in seconds
    duration = len(audio_segment) / 1000  # pydub returns duration in milliseconds
    # Transcribe audio to text using SpeechRecognition
    recognizer = sr.Recognizer()
    with sr.AudioFile(audio_file) as source:
        audio_data = recognizer.record(source)
        try:
            transcription = recognizer.recognize_google(audio_data)
        except sr.UnknownValueError:
            print("Speech Recognition could not understand audio")
            return None
        except sr.RequestError:
            print("Could not request results from Google Speech Recognition service")
            return None
    word_count = len(transcription.split())
    # Calculate pace in words per minute (WPM)
    minutes = duration / 60
    pace = word_count / minutes if minutes > 0 else 0
    # Determine pace category
    if 130 <= pace <= 160:
        pace_feedback = "Your pace is perfect."
    elif 100 <= pace < 130:
        pace_feedback = "You need to speed up a little bit."
    elif pace < 100:
        pace_feedback = "You are going very slow."
    elif 160 < pace <= 190:
        pace_feedback = "You need to slow down a little bit."
    else:  # pace > 190
        pace_feedback = "You are going very fast."
    return pace, transcription, pace_feedback
```

Figure 4.5: Pace Code

grammatical_score Function 4.6 Compares a spoken sentence with its corrected version to determine grammatical accuracy.

- Normalize both original and corrected text.
- Compare word-by-word differences.
- Count errors and compute error ratio.
- Assign a score (1-10) based on error ratio.

```python
def grammatical_score(original, corrected):
    # Normalize both the original and corrected sentences
    original_normalized = normalize_text(original)
    corrected_normalized = normalize_text(corrected)

    # Using difflib to compare the normalized sentences
    diff = difflib.ndiff(original_normalized.split(), corrected_normalized.split())
    changes = list(diff)

    # Count the number of words that changed (added/removed/modified)
    corrected_words = [word for word in changes if word.startswith('+ ')]
    removed_words = [word for word in changes if word.startswith('- ')]

    num_changes = len(corrected_words) + len(removed_words)
    total_words = len(original_normalized.split())

    # Calculate the error ratio (number of corrections divided by total words in the original text)
    error_ratio = num_changes / total_words if total_words > 0 else 0

    # Score based on the error ratio, normalizing it with respect to the length
    if error_ratio == 0:
        return 10  # Perfect grammar
    elif error_ratio < 0.05:
        return 9  # Very few errors (minor grammar issues)
    elif error_ratio < 0.1:
        return 8  # A few grammar issues
    elif error_ratio < 0.2:
        return 6  # Noticeable grammar issues
    else:
        return 4  # Many grammatical errors
```

Figure 4.6: Grammar Code

classify_posture Function 4.7 Detects body posture based on shoulder, hip, knee, and head positions.

- Extract keypoint coordinates.
- Determine standing, sitting, slouching, leaning positions.
- Return the identified posture type.

```
def classify_posture(keypoints):
    # Example: Extract keypoints for important joints
    shoulder_y = keypoints[5]['y']  # Right shoulder
    hip_y = keypoints[11]['y']  # Right hip
    knee_y = keypoints[13]['y']  # Right knee
    ankle_y = keypoints[15]['y']  # Right ankle
    head_y = keypoints[0]['y']  # Nose (head)
    # **Standing Posture**
    if shoulder_y < hip_y:
        return "Standing"
    # **Sitting Posture**
    elif shoulder_y > hip_y:
        return "Sitting"
    # **Slouching Posture**: Shoulders lower than hips and the spine is not straight.
    if shoulder_y > hip_y and knee_y > shoulder_y and ankle_y > knee_y:
        return "Slouching"
    # **Leaning Forward**: Head and shoulders leaning forward
    if head_y < shoulder_y and knee_y > hip_y:
        return "Leaning Forward"
    # **Head Down**: Head lower than shoulders
    if head_y > shoulder_y:
        return "Head Down"
    # **Leaning Back**: Shoulders higher than hips and knees
    if shoulder_y < hip_y and knee_y < hip_y and ankle_y < knee_y:
        return "Leaning Back"
    # **Arms on Hips Posture**: Shoulders and hips are in line but arms are placed on hips
    if abs(keypoints[7]['x'] - keypoints[8]['x']) < 0.1 and abs(keypoints[6]['x'] - keypoints[9]['x']) < 0.1:
        return "Arms on Hips"
    return "Unknown Posture"
```

Figure 4.7: Posture Code

classify_hand_gesture Function 4.8 Recognizes common hand gestures (Thumbs Up, Open Palm, Closed Fist, etc.).

- Extract finger landmarks.
- Check which fingers are extended or folded.
- Match predefined gestures.
- Return gesture type.

```
def classify_hand_gesture(hand_landmarks):
    landmarks = np.array([(lm.x, lm.y) for lm in hand_landmarks.landmark])
    # Finger landmarks indices
    THUMB_TIP, INDEX_TIP, MIDDLE_TIP, RING_TIP, PINKY_TIP = 4, 8, 12, 16, 20
    THUMB_IP, INDEX_DIP, MIDDLE_DIP, RING_DIP, PINKY_DIP = 3, 7, 11, 15, 19
    THUMB_MCP = 2
    def is_finger_extended(tip, dip):
        return landmarks[tip][1] < landmarks[dip][1]  # y-coordinate: lower is higher
    thumb_extended = landmarks[THUMB_TIP][1] < landmarks[THUMB_IP][1]
    index_extended = is_finger_extended(INDEX_TIP, INDEX_DIP)
    middle_extended = is_finger_extended(MIDDLE_TIP, MIDDLE_DIP)
    ring_extended = is_finger_extended(RING_TIP, RING_DIP)
    pinky_extended = is_finger_extended(PINKY_TIP, PINKY_DIP)
    # Gesture classification logic
    if all([index_extended, middle_extended, ring_extended, pinky_extended]) and not thumb_extended:
        return "Open Palm"
    if not any([index_extended, middle_extended, ring_extended, pinky_extended, thumb_extended]):
        return "Closed Fist"
    if index_extended and not any([middle_extended, ring_extended, pinky_extended]):
        return "Pointing Finger"
    if thumb_extended and not any([index_extended, middle_extended, ring_extended, pinky_extended]):
        return "Thumbs Up"
    if not thumb_extended and not any([index_extended, middle_extended, ring_extended, pinky_extended]):
        return "Thumbs Down"
    if not thumb_extended and all([index_extended, middle_extended]) and not any([ring_extended, pinky_extended]):
        return "Victory Sign"
    if thumb_extended and index_extended and not any([middle_extended, ring_extended, pinky_extended]):
        return "OK Sign"
    if index_extended and pinky_extended and not any([middle_extended, ring_extended]):
        return "Rock Sign"
    if thumb_extended and pinky_extended and not any([index_extended, middle_extended, ring_extended]):
        return "Call Me"
    return "Unknown Gesture"
```

Figure 4.8: Gesture Code

Facial Emotion Detection Model

The model 4.9 The DeepFace Emotion Recognition Model is built using a pre-trained Convolutional Neural Network (CNN) architecture, fine-tuned specifically for facial emotion detection. It leverages the pre-trained VGG-Face and ResNet models for feature extraction, which have been trained on large facial recognition datasets like AffectNet and FER-2013.

- **Emotion Refinement:** The detected emotion is refined based on psychological compatibility with eye contact.

```
You, 24 hours ago | 1 author (You)
import cv2
import os
import numpy as np
from collections import Counter
from deepface import DeepFace

def refine_emotion_prediction(emotion, eye_contact):


    # Emotion and Eye Contact compatibility based on psychological studies

    if emotion == "neutral":
        # Neutral emotions might have less distinct eye contact, both with or without eye contact is possible
        if eye_contact == "Eye Contact":
            return "attentive"
        else:
            return "indifferent"  # No strong emotion, could show less engagement

    if emotion == "angry":
        # Angry emotions are often associated with direct eye contact
        if eye_contact == "Eye Contact":
            return "intense"  # Direct eye contact with anger intensifies the emotion
        else:
            return "defensive"  # Avoiding eye contact in anger could show defensiveness

    if emotion == "fear":
        # Fear is typically linked to avoiding eye contact (avoidance behavior)
        if eye_contact == "Eye Contact":
            return "nervous"  # Fearful but maintaining eye contact can show nervousness or anxiety
        else:
            return "distrust"  # Fear often leads to avoiding eye contact, showing discomfort or distrust

    if emotion == "happy":
        # Happiness is often associated with more relaxed and positive body language, which can include eye contact
        if eye_contact == "Eye Contact":
            return "joyful"  # Happiness with eye contact indicates engagement and connection
        else:
```

Figure 4.9: DeepFace Emotion Recognition Model

- **Emotion Detection:** The code uses DeepFace.analyze to predict emotions from facial images.

- **Eye Contact Detection:** OpenCV's Haar Cascade classifier is used to detect whether the subject in the frame is making eye contact.

- **Final Prediction:** The code combine emotions prediction and eye contact status to provide a refined emotion, improving accuracy by considering non-verbal cues like eye contact.

```python
def detect_eye_contact(image):
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    eyes_cascade = cv2.CascadeClassifier(cv2.data.haarcascades + 'haarcascade_eye.xml')
    eyes = eyes_cascade.detectMultiScale(gray, 1.3, 5)
    if len(eyes) > 0:
        return "Eye Contact"
    else:
        return "No Eye Contact"

def preprocess_frame(image):
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    face_cascade = cv2.CascadeClassifier(cv2.data.haarcascades + 'haarcascade_frontalface_default.xml')
    faces = face_cascade.detectMultiScale(gray, 1.1, 4)

    if len(faces) > 0:
        x, y, w, h = max(faces, key=lambda rect: rect[2] * rect[3])
        cv2.rectangle(image, (x, y), (x + w, y + h), (0, 255, 0), 2)
        cropped_face = image[y:y + h, x:x + w]
        return image
    return image

def predict_emotions_on_frames(frame):
    processed_image = preprocess_frame(frame)
    analysis = DeepFace.analyze(img_path=processed_image, actions=['emotion'], enforce_detection=False)
    emotion = analysis[0]['dominant_emotion']

    eye_contact = detect_eye_contact(processed_image)
    refined_emotion = refine_emotion_prediction(emotion, eye_contact)
    return refined_emotion, eye_contact
```

Figure 4.10: DeepFace Emotion Recognition Model

Tone Emotion Detection Model 4.11

- **Training the Model:** The model is trained using the `fit()` method.
- **Input Data:**
  - `x_train`: The training dataset (reshaped for CNN-LSTM).
  - `y_train`: The target labels, one-hot encoded for classification.
- **Number of Epochs:**
  - `epochs=50`: The model will iterate over the dataset up to 50 times.
- **Validation Data:**
  - `validation_data=(x_test, y_test)`: Used to monitor model performance after each epoch.
- **Batch Size:**

  - `batch_size=64`: The model updates weights after processing 64 samples at a time.

- **Callbacks:** Functions to enhance training:

  - `early_stop`: Stops training if no improvement is detected to prevent overfitting.
  - `lr_reduction`: Lowers the learning rate when the training progress slows.
  - `model_checkpoint`: Saves the best model weights during training.

```
# Train the model using the fit method
history = model.fit(
    x_train,  # Input training data (reshaped for CNN-LSTM architecture)
    y_train,     # Corresponding one-hot encoded target labels for the training data
    epochs=50,   # Maximum number of epochs to train the model
    validation_data=(x_test, y_test),  # Validation data used to evaluate the model during training
    batch_size=64,  # Number of samples per gradient update (batch size)
    callbacks=[early_stop, lr_reduction, model_checkpoint]  # List of callbacks to enhance training
)
```

```
Epoch 1/50
609/609 [==============================] - 116s 176ms/step - loss: 1.4134 - accuracy: 0.4612 - val_loss: 1.5701 - val_accuracy: 0.4272
Epoch 2/50
609/609 [==============================] - 106s 174ms/step - loss: 1.1311 - accuracy: 0.5590 - val_loss: 1.1125 - val_accuracy: 0.5736
Epoch 3/50
609/609 [==============================] - 106s 175ms/step - loss: 0.9911 - accuracy: 0.6172 - val_loss: 1.2060 - val_accuracy: 0.5508
Epoch 4/50
609/609 [==============================] - 106s 174ms/step - loss: 0.8980 - accuracy: 0.6541 - val_loss: 1.1263 - val_accuracy: 0.5845
Epoch 5/50
609/609 [==============================] - 106s 174ms/step - loss: 0.7886 - accuracy: 0.6979 - val_loss: 0.8593 - val_accuracy: 0.6742
Epoch 6/50
609/609 [==============================] - 106s 174ms/step - loss: 0.6691 - accuracy: 0.7490 - val_loss: 0.9310 - val_accuracy: 0.6601
Epoch 7/50
609/609 [==============================] - 106s 174ms/step - loss: 0.5480 - accuracy: 0.7971 - val_loss: 0.6999 - val_accuracy: 0.7398
Epoch 8/50
609/609 [==============================] - 106s 174ms/step - loss: 0.4135 - accuracy: 0.8510 - val_loss: 0.5057 - val_accuracy: 0.8207
Epoch 9/50
609/609 [==============================] - 106s 175ms/step - loss: 0.3312 - accuracy: 0.8807 - val_loss: 0.4205 - val_accuracy: 0.8527
Epoch 10/50
609/609 [==============================] - 106s 174ms/step - loss: 0.2301 - accuracy: 0.9197 - val_loss: 0.4459 - val_accuracy: 0.8460
Epoch 11/50
609/609 [==============================] - 106s 174ms/step - loss: 0.2040 - accuracy: 0.9271 - val_loss: 0.4964 - val_accuracy: 0.8363
Epoch 12/50
```

Figure 4.11: Tone code

Pronunciation Quality Function 4.12

Detects pronunciation quality using a pre-trained pronunciation model that returns a score.

- Determines confidence scores for each pronunciation level (Beginner, Intermediate, Proficient, and Advanced).
- Assigns weights to each level.
- Calculates and returns the overall pronunciation score.

pronounciation.py 3

Presentation-Skills > pronounciation.py > ...

```python
# Load the audio
waveform, sample_rate = torchaudio.load(audio_file)

# Convert stereo to mono if needed
if waveform.shape[0] > 1:
    waveform = waveform.mean(dim=0, keepdim=True)

# Resample to 16kHz if needed
if sample_rate != 16000:
    resample = T.Resample(orig_freq=sample_rate, new_freq=16000)
    waveform = resample(waveform)

# Normalize the waveform
waveform = (waveform - waveform.mean()) / waveform.std()

# Convert tensor to numpy array
waveform = waveform.squeeze(0).numpy()

# Run prediction
result = pipe(waveform)

# Define score mapping
score_mapping = {
    "advanced": 100,
    "proficient": 75,
    "intermediate": 50,
    "beginer": 25
}

# Compute final score
final_score = sum(score_mapping[item["label"]] * item["score"] for item in result)

# Print result
print(f"Pronunciation Score: {final_score:.2f} / 100")
```

Figure 4.12: Pronunciation code

```python
def evaluate(model, val_loader, criterion, device):
        for batch in val_loader:
            inputs, labels = batch
            inputs, labels = inputs.to(device), labels.to(device)

            outputs = model(inputs).logits
            loss = criterion(outputs, labels)
            total_loss += loss.item()

            preds = torch.argmax(outputs, dim=1)
            correct += (preds == labels).sum().item()

    accuracy = correct / len(val_loader.dataset)
    return total_loss / len(val_loader), accuracy
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.
📌 Epoch 1: Val Loss = 0.5750 | Val Acc = 0.8143
Some weights of Wav2Vec2ForSequenceClassification were not initialized from the model checkpoint at facebook/wav2vec2-base and a
er.bias', 'classifier.weight', 'projector.bias', 'projector.weight']
You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.
📌 Epoch 2: Val Loss = 0.5116 | Val Acc = 0.8279
Some weights of Wav2Vec2ForSequenceClassification were not initialized from the model checkpoint at facebook/wav2vec2-base and a
er.bias', 'classifier.weight', 'projector.bias', 'projector.weight']
You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.
📌 Epoch 3: Val Loss = 0.4751 | Val Acc = 0.8436
Some weights of Wav2Vec2ForSequenceClassification were not initialized from the model checkpoint at facebook/wav2vec2-base and a
er.bias', 'classifier.weight', 'projector.bias', 'projector.weight']
You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.
📌 Epoch 4: Val Loss = 0.4383 | Val Acc = 0.8611
Some weights of Wav2Vec2ForSequenceClassification were not initialized from the model checkpoint at facebook/wav2vec2-base and a
er.bias', 'classifier.weight', 'projector.bias', 'projector.weight']
You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.
📌 Epoch 5: Val Loss = 0.4248 | Val Acc = 0.8568
```

Figure 4.13: Filler word model fine-tunning

Fine-tunning for Filler Word detector 4.13

1400 labeled 2 second audio files pre-processed and used to fine tune the pre-trained wave model for 5 epochs

- Pre-processing by labeling and converting the audio files into appropriate format.
- Trains the model with the processed data and evaluates results.
- Used the model on long audio files and it turned out to be very accurate and precise.

testfiller.py 2

Presentation-Skills > testfiller.py > ...

```python

    # Skip segments shorter than 2 seconds (last segment)
    if segment.shape[1] < samples_per_segment:
        break

    # Convert segment to model input
    inputs = processor(segment.squeeze().numpy(), sampling_rate=16000, return_tensors="pt", padding=True)

    # Get model predictions
    with torch.no_grad():
        logits = model(inputs.input_values).logits
        probabilities = F.softmax(logits, dim=-1)
        predictions = torch.argmax(probabilities, dim=-1)

    # Count filler words
    for pred in predictions:
        label = label_map[pred.item()]
        if label in filler_counts:
            filler_counts[label] += 1

# Compute filler rate
total_fillers = filler_counts["Uh"] + filler_counts["Um"]
filler_rate = total_fillers / total_segments if total_segments > 0 else 0

# Convert to a 100-point score
score = max(100 - (filler_rate * 100), 0)

# Generate feedback
if score > 90:
    feedback = "Excellent! You used very few filler words. Keep it up! 🎯"
elif score > 75:
    feedback = "Good job! You have some fillers, but it's not distracting. Try to be more mindful. 😊"
elif score > 50:
    feedback = "You're using filler words often. Consider pausing instead of saying 'Uh' or 'Um'. 😉"
else:
    feedback = "High use of filler words detected! Try slowing down and structuring your thoughts before speaking. 🚀"
```

Figure 4.14: Filler Word code

Filler Word detector Function 4.14

Detects filler word then calculates a score and returns feedback by using a fine tuned pre-trained model by processing audio into 2 second chunks and passes these files.

- Loops the original audio into 2 second audio clips and passes them to the model.
- The model detects the number of filler words very accurately.
- Calculates and returns score and gives appropriate feedback.

4.2.6 Interaction viewpoint

User Sequence Diagram 4.16

- The User signs up/logs in via the Mobile App.
- After authentication, the User uploads a presentation video.
- The Preprocessing Module extracts speech, gestures, and facial expressions.
- The AI Processing Module analyzes speech, emotion, and body language.
- The Feedback System stores and compiles the results.
- The final feedback and rating are sent to the Mobile App.
- The User views the detailed feedback and ratings.

AI Sequence Diagram 4.17

- The AI Processing Module analyzes the uploaded video.
- The NLP Module processes speech for clarity, grammar, and filler words.
- The Speaker Interaction Module assesses confidence and audience engagement.
- The Emotion Detection Module evaluates emotional tone.
- The Body Language Processing Module detects posture, gestures, and facial expressions.
- The AI Processing Module compiles insights and generates a rating.
- The Feedback System receives the final analysis results.

Admin Sequence Diagram 4.18

- The Admin logs in via the Admin Panel.
- After authentication, the Admin can view user reports.
- The Admin Panel fetches user reports from the Database.
- The Database returns the requested reports.
- The Admin Panel displays the reports to the Admin.
- The Admin manages user accounts (e.g., updating user data).
- The Admin Panel updates the user data in the Database.
- The Database confirms the successful update.

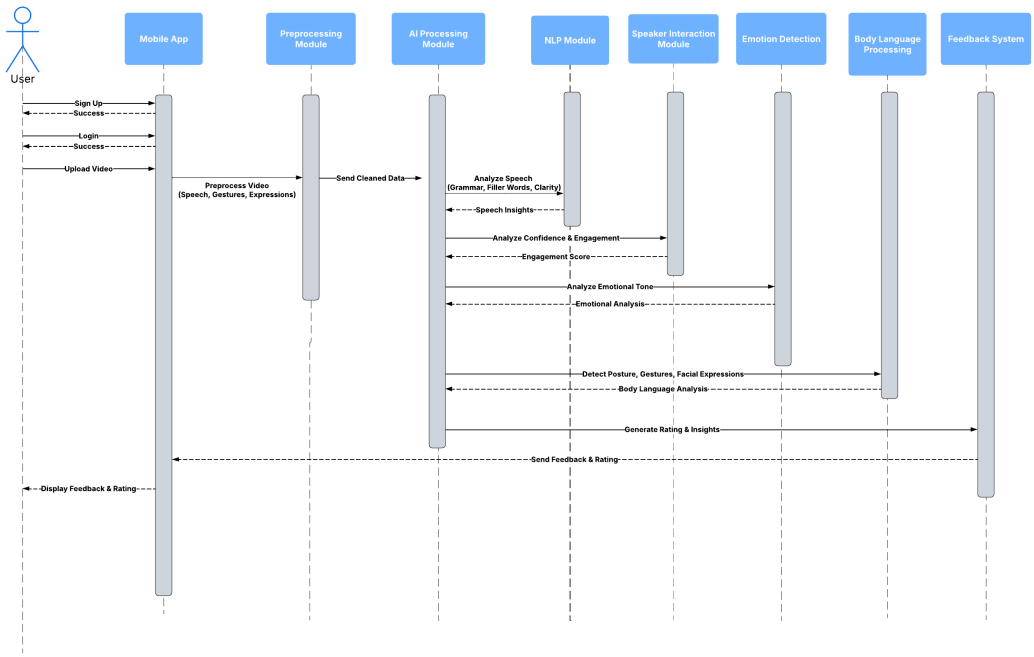

Figure 4.15: Sequence Diagram

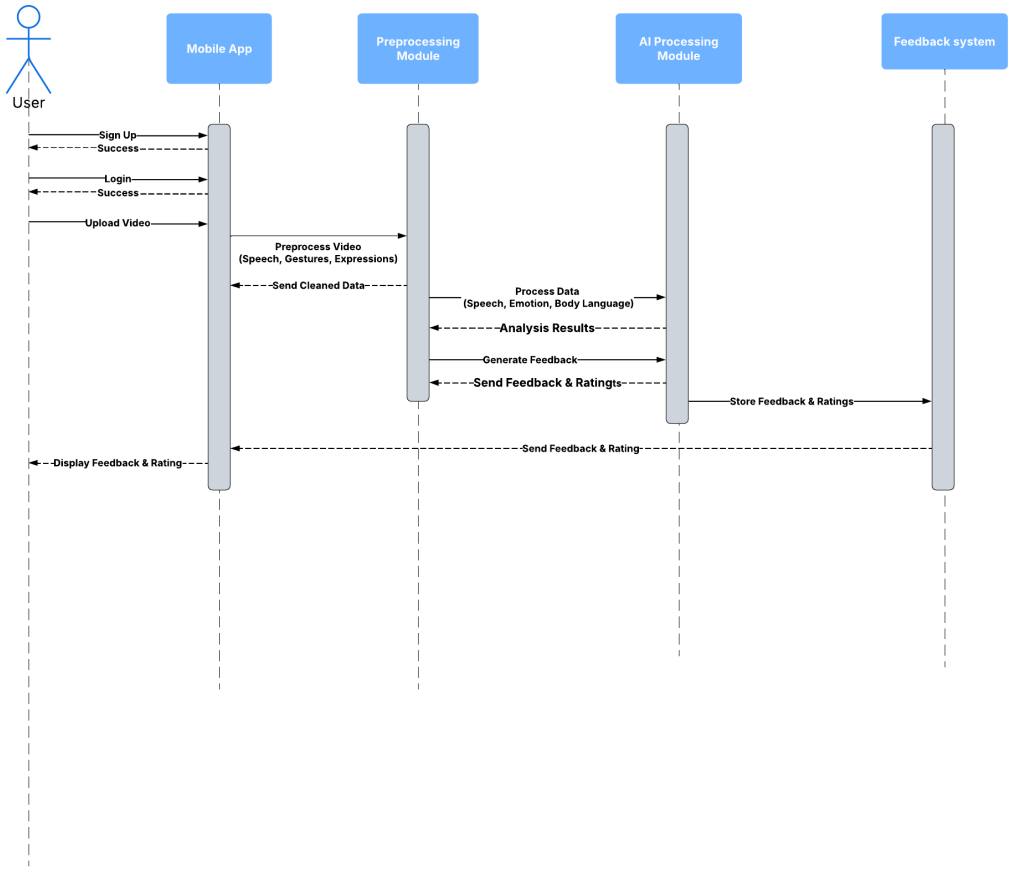

Figure 4.16: User Sequence Diagram

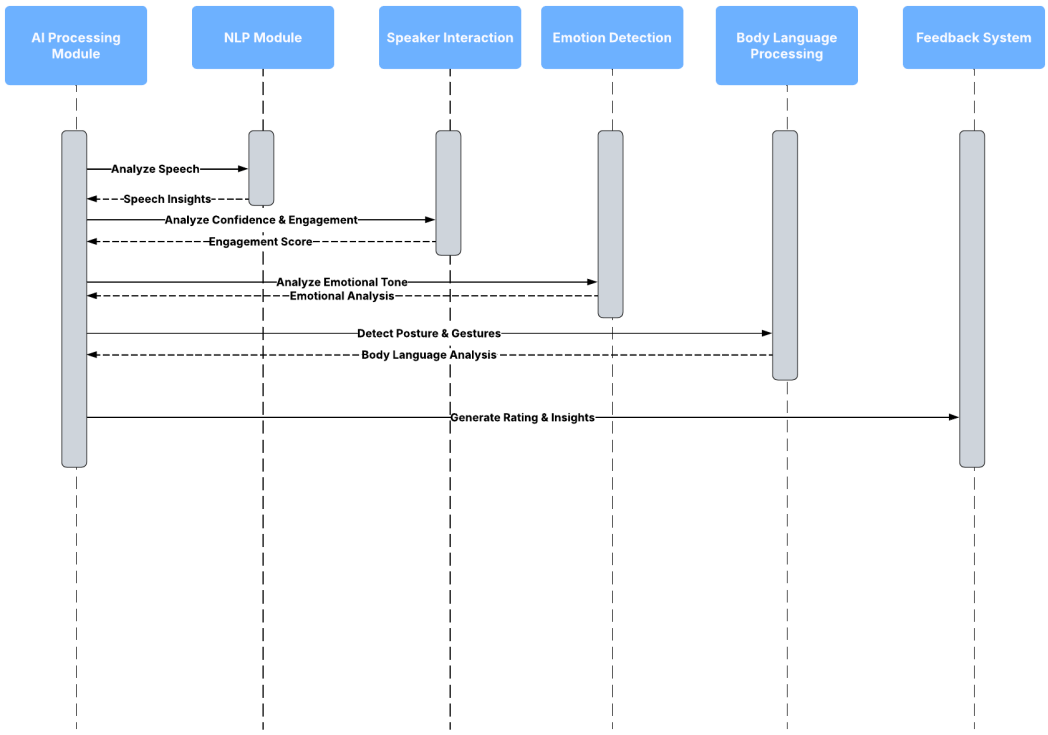

Figure 4.17: AI Sequence Diagram

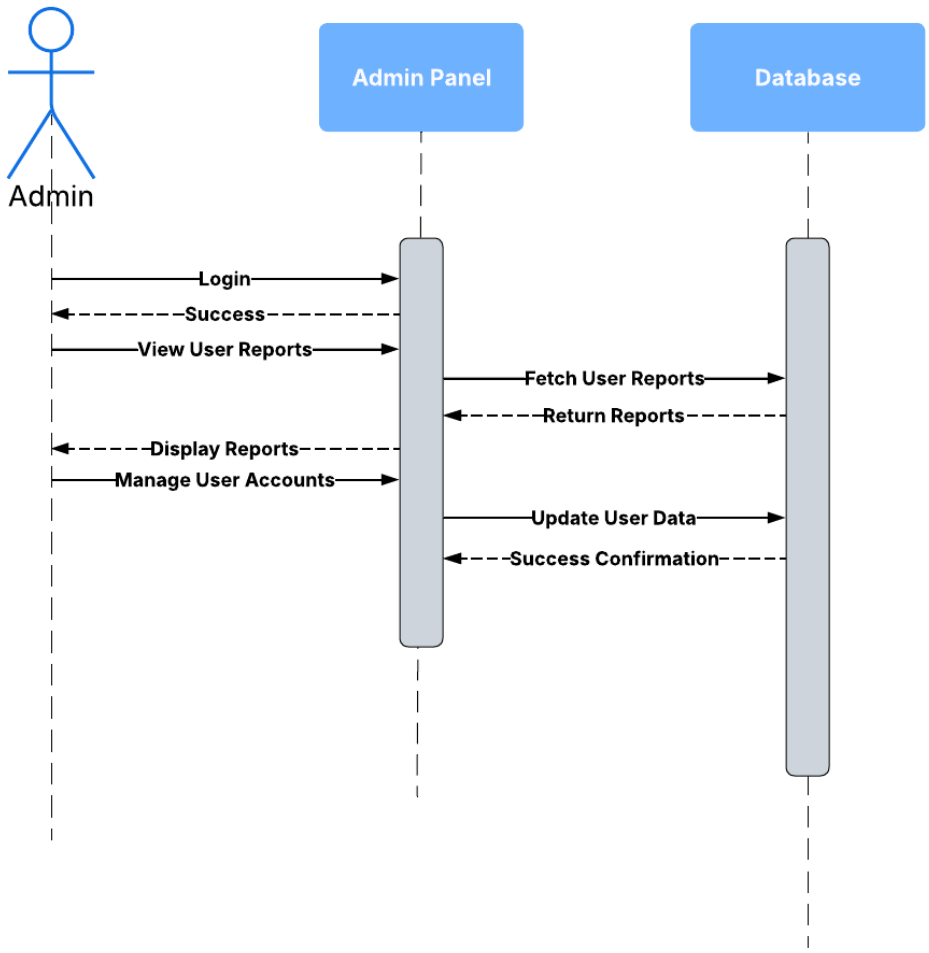

Figure 4.18: Admin Sequence Diagram

4.2.7 Interface viewpoint

User Interface (UI):

- Allows users to sign up and log in
- Retrieves AI-generated feedback and displays it to the user.

Backend API (Processing Module):

- Acts as a bridge between the UI and AI modules.
- Handles text processing, grammar analysis, sentiment detection, and writing style evaluation.
- Compiles analysis results for feedback generation.

AI Processing Modules:

- NLP Module: Analyzes grammar, clarity, and fluency.
- Sentiment Analyzer: Detects emotional tone and engagement.

Feedback System:

- Aggregates AI-generated insights into a structured feedback report.
- Provides performance scores and improvement recommendations.

4.3 Data Design

4.3.1 Data Description

Tone Detection (Audio)

The system utilizes four primary datasets for tone emotion analysis: SAVEE, TESS, RAVDESS, and CREMA-D. These datasets are combined to create a robust and comprehensive model capable of accurately detecting and analyzing emotional tones in speech.

- SAVEE serves as a foundational dataset, providing audio-visual recordings of emotional speech. The dataset includes recordings from speakers expressing various emotions in controlled environments, ensuring clarity and consistency in the data. Preprocessing steps for SAVEE involve audio normalization and segmentation to prepare the data for effective model training.

- TESS contributes a diverse range of emotional speech samples, recorded from multiple speakers to capture natural variations in tone and expression. The dataset is structured to cover key emotional categories, making it an excellent resource for enhancing the model's ability to generalize across different voice profiles. Preprocessing includes audio feature extraction and standardization to ensure consistency with other datasets.

- RAVDESS, known for its high-quality recordings, adds a rich variety of emotional expressions through both speech and song. This dataset introduces controlled variability in pitch, tone, and rhythm, which is crucial for developing a nuanced understanding of emotional cues in speech. Preprocessing steps for RAVDESS involve noise reduction, feature scaling, and alignment with the model's input requirements.

- CREMA-D offers a wide range of emotional speech data sourced from diverse speakers, enhancing the model's robustness in real-world scenarios. The dataset captures subtle emotional variations, providing valuable data for fine-tuning the model's sensitivity to different emotional intensities. Preprocessing includes normalization, feature extraction, and data augmentation to improve model performance.By integrating SAVEE, TESS, RAVDESS, and CREMA-D, the system benefits from a balanced combination of con-

trolled and natural emotional speech data. This comprehensive approach ensures the model can effectively recognize and analyze emotional tones, improving its application in diverse environments and real-world scenarios.

Facial Expressions

The system utilizes two primary datasets for facial expression recognition: FER2013 and AffectNet.

- FER2013 4.19 serves as both the training and validation dataset, while AffectNet is incorporated for additional training and validation to enhance model robustness.FER2013 contains grayscale images labeled with different emotion categories. The dataset is structured into distinct subsets for training, validation, and testing. It provides a well-balanced distribution of facial expressions, making it an ideal dataset for training deep learning models. Preprocessing steps include image normalization, data augmentation, and structured storage to ensure the model generalizes well to unseen data.

Figure 4.19: FER dataset sample

- AffectNet 4.20, a significantly larger dataset, consists of images representing a wide range of facial expressions. In addition to categorical emotion labels, it includes valence-arousal values that quantify the intensity of emotions. The dataset is divided into training and validation sets, allowing for more comprehensive model evaluation. Since AffectNet consists of high-resolution RGB images, preprocessing steps involve resizing, grayscale conversion (when needed), and standardizing emotion labels to align with FER2013. The inclusion of AffectNet introduces greater variability in facial expressions, improving the model's ability to recognize emotions in diverse real-world scenarios.By integrating both

datasets, the system benefits from a combination of controlled and real-world facial expression data, ensuring more accurate emotion analysis for evaluating presentation skills.

Figure 4.20: AffectNet dataset sample

Filler Words

- The PodcastFillers dataset is used to enhance the analysis of filler words and speech disfluencies in natural spoken language. It includes 199 podcast episodes with annotations for filler words like "uh" and "um," along with non-verbal sounds such as breaths and laughter.The dataset offers diverse speech samples from over 350 speakers, ensuring robust model performance across different speaking styles. It includes speech-to-text transcripts, annotation files, and voice activity detection (VAD) predictions.Preprocessing involves audio normalization, segmentation, and feature extraction to improve filler word detection. This real-world data helps develop models for applications like conversational AI, transcription services, and speech analytics.

4.3.2 Dataset Description

Table 4.7: (SAVEE) Dataset Description

Dataset Name	Surrey Audio-Visual Expressed Emotion (SAVEE)
Link	`https://www.kaggle.com/datasets/ejlok1/surrey-audiovisual-expressed-emotion-savee`
Size	200 MB
Number of Classes	7
Audio Format	WAV
Number of Audio Files	480
Notes	Contains recordings from four male speakers expressing emotions in a controlled setting.

Table 4.8: (TESS) Dataset Description

Dataset Name	Toronto Emotional Speech Set (TESS)
Link	`https://www.kaggle.com/datasets/ejlok1/toronto-emotional-speech-set-tess`
Size	1.3 GB
Number of Classes	7
Audio Format	WAV
Number of Audio Files	2,800
Notes	Recorded by two female speakers , with sentences spoken in different emotions.

Table 4.9: (RAVDESS) Dataset Description

Dataset Name	Ryerson Audio-Visual Database of Emotional Speech and Song (RAVDESS)
Link	`https://www.kaggle.com/datasets/uwrfkaggler/ravdess-emotional-speech-audio`
Size	1.3 GB
Number of Classes	7
Audio Format	WAV
Number of Audio Files	1,440
Notes	Includes emotional speech and song recordings from professional actors

Table 4.10: (CREMA-D Dataset Description)

Dataset Name	Crowd-Sourced Emotional Multimodal Actors Dataset (CREMA-D)
Link	`https://www.kaggle.com/datasets/ejlok1/cremad`
Size	744 MB
Number of Classes	6 (Angry, Disgust, Fear, Happy, Neutral, Sad)
Audio Format	WAV
Number of Audio Files	7,438
Notes	Contains recordings from 91 actors (48 male, 43 female), with six emotions captured through speech.

Table 4.11: FER2013 Dataset Description

Dataset Name	FER2013
Link	`https://www.kaggle.com/datasets/msambare/fer2013`
Size	30 MB
Number of classes	8
Image Dimensions	48×48 pixels
Number of images	35,887
Image file size	2 KB
Notes	The dataset is divided into training , validation, and testing).

Table 4.12: AffectNet Dataset Description

Dataset Name	AffectNet
Link	`http://mohammadmahoor.com/affectnet/`
Size	120 GB
Number of classes	8
Image Dimensions	224×224 pixels
Number of images	287,401
Image file size	50 KB to 300 KB
Notes	More computationally expensive due to higher image resolution.

Table 4.13: PodcastFillers Dataset Description

Dataset Name	PodcastFillers
Link	`https://podcastfillers.github.io/dataset/`
Size	145 hours of audio data
Annotations	85,803 manually annotated audio events
Audio Format	MP3
Notes	Pre-arranged into train, validation, and test sets for machine learning applications

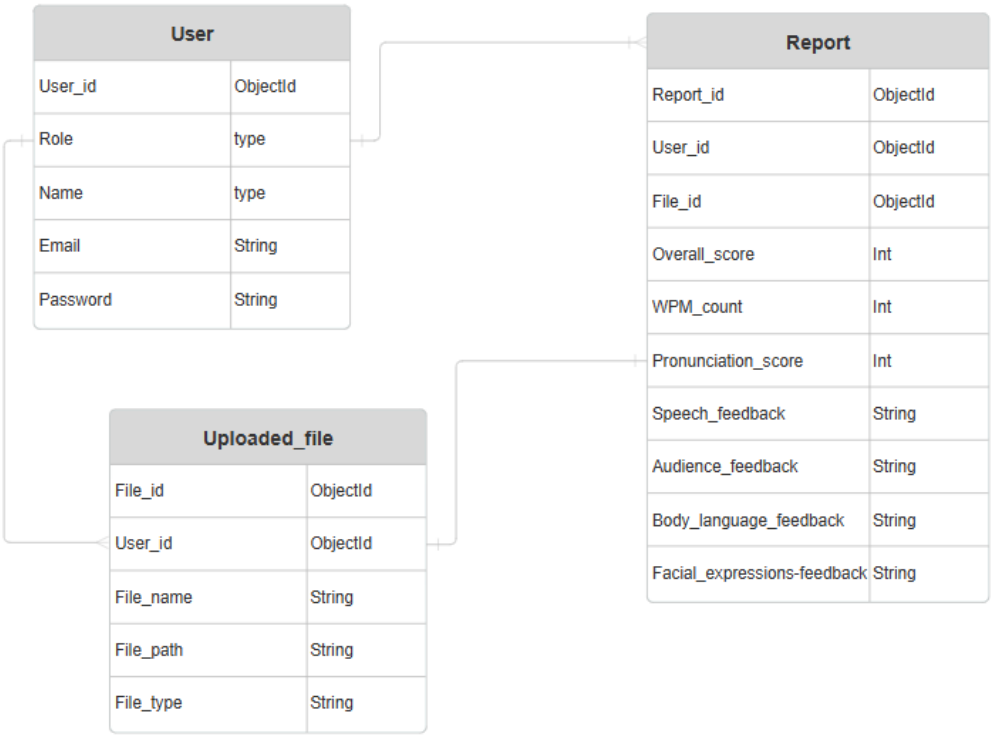

Figure 4.21: NOSQL Schema Diagram

4.3.3 Database Design

Our database is the noSQL database.The schema shown in figure 4.21 represents our database design.The database contains the user and the uploaded speech and the corresponding report for each speech.

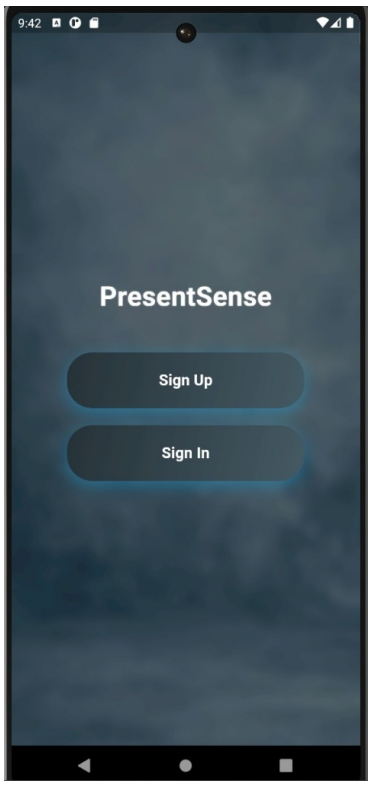

Figure 4.22: Sign Up/Login

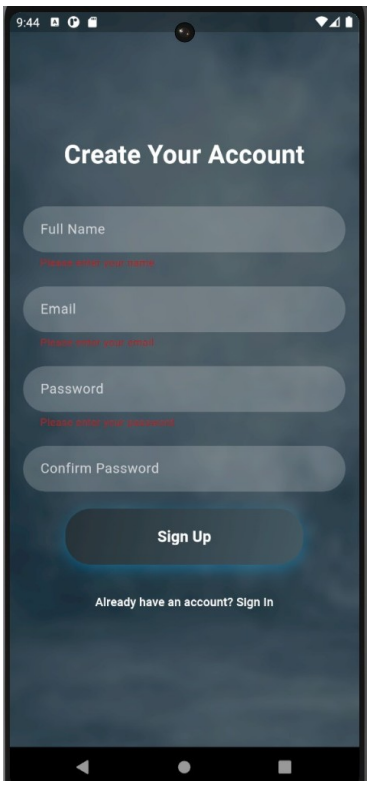

Figure 4.23: Sign Up

4.4 Human Interface Design

4.4.1 User Interface

After the user signs up and logs in he will be able to upload videos of a speech.The system analyzes different aspects of the speech and returns a detailed report about verbal and non-verbal cues of the speech. The user shall also be able to access previous reports.

4.4.2 Screen Images

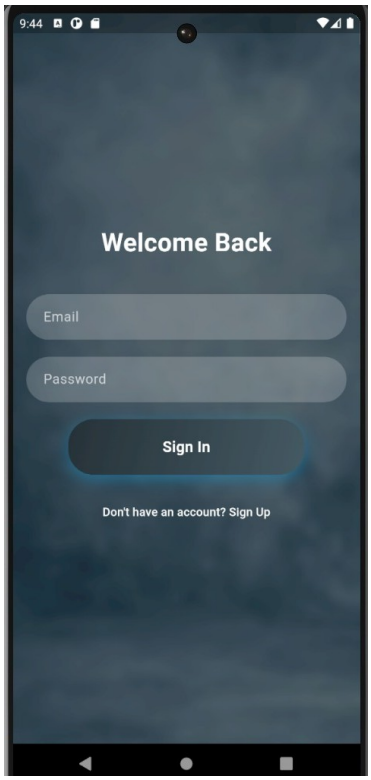

Figure 4.24: Login

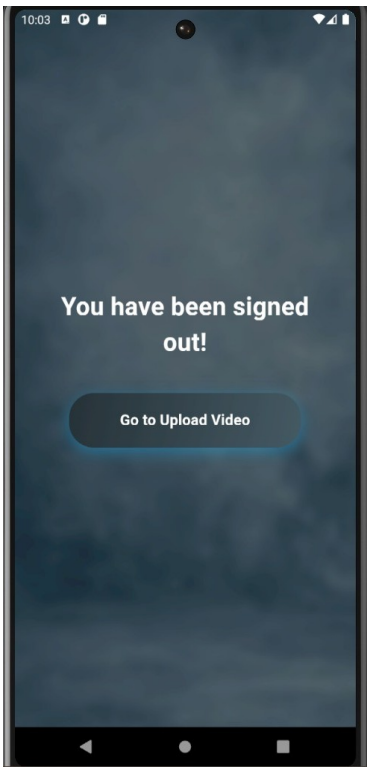

Figure 4.25: Signed Out

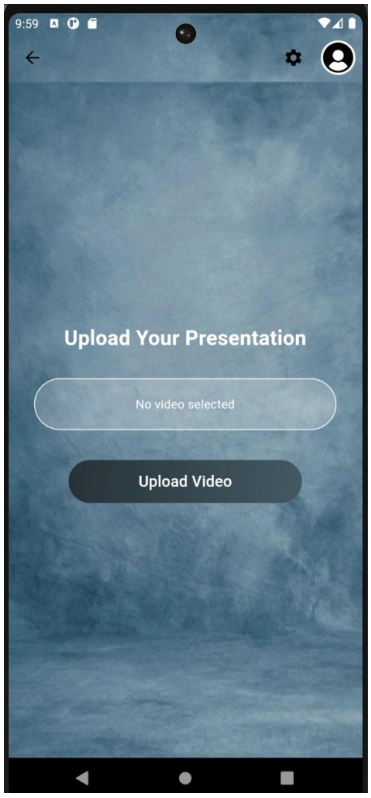

Figure 4.26: Upload

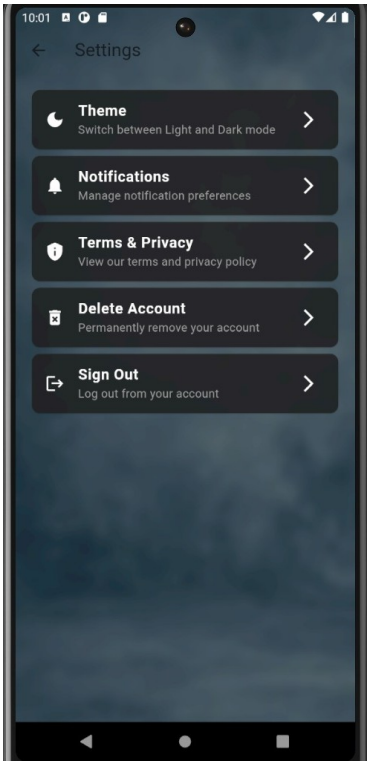

Figure 4.27: Settings

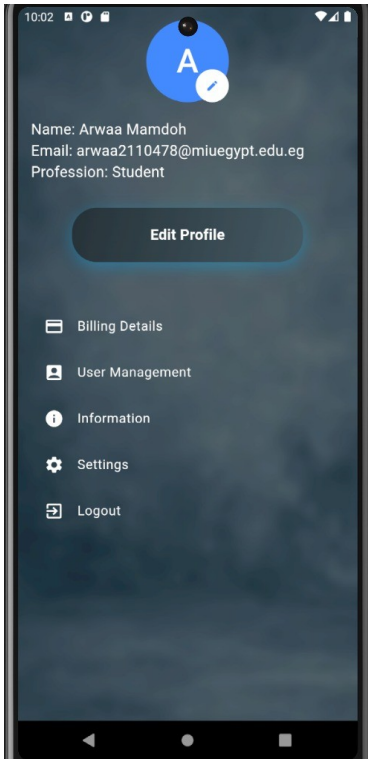

Figure 4.28: Profile

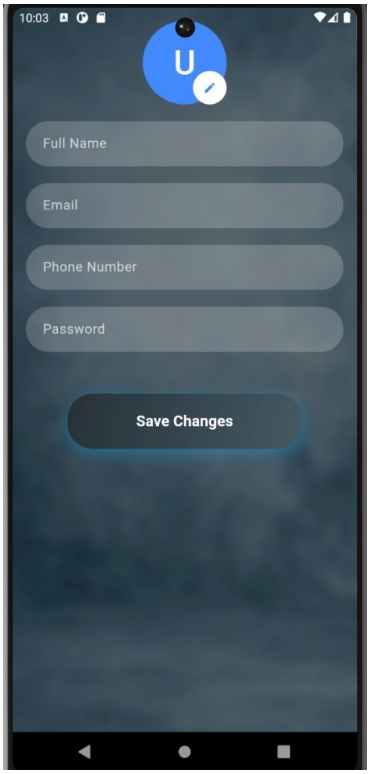

Figure 4.29: Edit

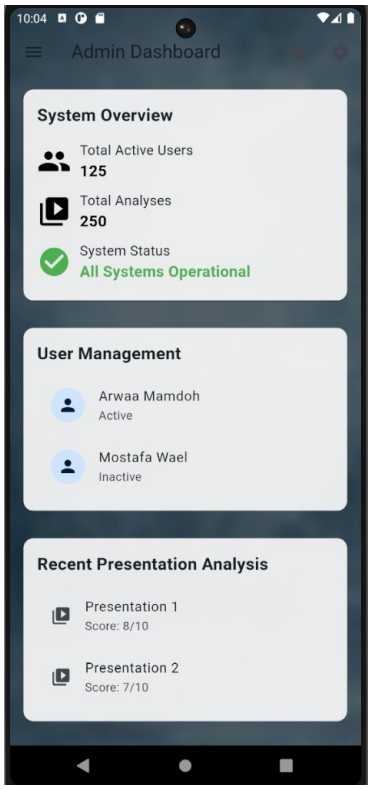

Figure 4.30: Admin Dashboard

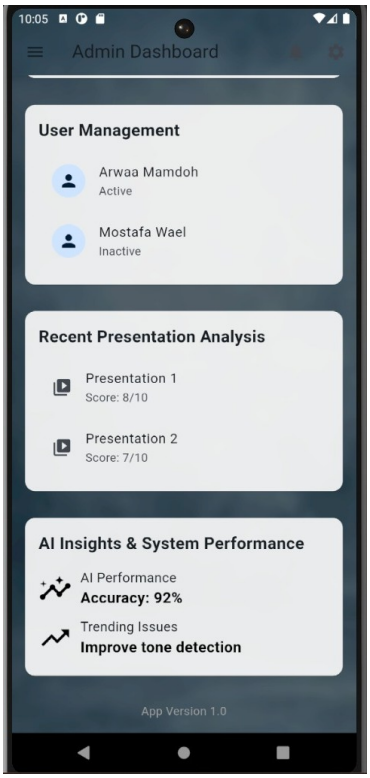

Figure 4.31: Admin Dashboard

4.4.3 Screen Objects and Actions

- The button Sign up redirects the user and allows him to insert his data. The sign up button in this page officially creates the account.
- If the user has already created an account, they just use their credentials to sign in and start using the functionalities they need.
- After signing in, the user will enter the home page that will allow him to start uploading speech samples by redirecting him to the drive to select a desired sample.
- The user will also be able to access the settings page by clicking on the icon.
- The user will also be able to access his profile where he can view his details and be able to update his details by clicking on the edit profile button where he can finalize the changes by clicking on the save changes button.
- As an admin signs in, he will automatically access the admin dashboard where he can view recent valuable statistics.

4.5 Implementation

Containing a comprehensive description of the implementation of your software, including the language(s) and platform chosen, problems encountered, any changes made to the design as a result of the implementation, etc.

Programming Languages

- **Python:** Used for developing and fine-tuning AI models for speech and body language analysis, as well as backend processing.
- **Flutter:** Used for building the mobile application, providing a smooth and responsive user experience.

Platforms Used

- **MongDB:** A NoSQL database is used to store user data, reports, and analysis results.

- **Cloud Storage:** Used to store and manage user-uploaded video files efficiently.
- **Backend API Server:** Handles requests from the frontend and connects to AI models for analysis.
- **AI Model Hosting:** Runs AI models for speech and body language analysis, either on a cloud service or a self-hosted server.

4.6 Testing Plan

4.6.1 Test Scenario X

Test Cases

The test cases 4.14 represent the functionality of the speech analysis program under various scenarios. As the user uploads different audio files, the system checks for pronunciation, filler words, pace, and grammatical issues.

Table 4.14: Test Cases for Scenario X

Test Case ID	**Test Case Description**	**Functional Req Code**	**Test Data**	**Expected Results**
TC06	User uploads a speech file	FR12	Valid file	System accepts and processes the file
TC07	User uploads an unsupported file format	FR12	Invalid file	System rejects file, shows error message

Test Case ID	**Test Case Description**	**Functional Req Code**	**Test Data**	**Expected Results**
TC08	User has good pronunciation	FR07	Speech sample	System detects pronunciation issues and returns a low score
TC09	User has poor pronunciation	FR07	Speech sample	System detects proficiency in pronunciation and returns a high score
TC10	User speaks slowly	FR05	Speech sample	System detects pace and suggests an increase in pace
TC11	User speaks with appropriate pace	FR05	Speech sample	System detects pace and suggests that pacing is fine
TC12	User speaks quickly	FR05	Speech sample	System detects pace and suggests slowing down
TC13	User speaks with different emotional tones	FR09	Speech sample	System classifies emotion as anger, sad, fear, disgust, happy, or neutral

Test Case ID	**Test Case Description**	**Functional Req Code**	**Test Data**	**Expected Results**
TC14	System analyzes body language	FR10	Speech sample	System classifies body language as confidence, lack of confidence, relaxed, or interested
TC15	System recognizes hand gestures	FR11	Speech sample	System classifies hand gestures as friendly, confident, honest, determined, or encouraging
TC16	User has poor grammar	FR06	Speech sample	System detects grammatical errors and returns a low grammatical score
TC17	User has good grammar	FR06	Speech sample	System detects few grammatical errors and returns a high grammatical score
TC18	User has poor fluency and uses filler words	FR06	Speech sample	System detects filler words and gives feedback
TC19	Model generates a report	FR06	Speech sample	System provides appropriate feedback

Test Case ID	**Test Case Description**	**Functional Req Code**	**Test Data**	**Expected Results**
TC20	User accesses history of previous reports	FR14	User request	System retrieves and displays past reports and analysis
TC21	User requests support from the admin	FR15	User request	System forwards request to the admin panel
TC22	User deletes their account permanently	FR16	User request	System removes all user data and account
TC23	Admin edits a user profile	FR17	Admin request	System updates user profile information
TC24	Admin assists users with technical issues	FR18	User request	Admin provides troubleshooting or resolution
TC25	Admin views user feedback	FR19	User feedback data	System displays user feedback to the admin
TC26	Admin reviews uploaded presentations	FR20	User-uploaded file	System allows admin to review and manage user submissions

Test Case ID	Test Case Description	Functional Req Code	Test Data	Expected Results
TC27	Admin monitors user progress reports	FR21	User data	System generates performance tracking reports for users

4.6.2 Test Scenario Y

Test Cases

The test cases 4.15 focus on ensuring that the login and signup features are functioning correctly, covering both valid and invalid input scenarios.

Table 4.15: Test Cases for Scenario Y

Test Case ID	Test Case Description	Functional Req Code	Test Data	Expected Results
TC01	User signs up with valid details.	FR01	Name, valid password, valid email	System successfully creates a new user account.
TC02	User attempts to sign up with an invalid email format.	FR03	Name, valid password, invalid email	System displays an error message and rejects signup.
TC03	User attempts to sign up with an invalid password.	FR04	Name, invalid password, valid email	System displays an error message and rejects signup.
TC04	User logs in with correct credentials.	FR02	Valid email and password	User successfully logs into their account.
TC05	User logs in with an incorrect password.	FR02	Valid email, incorrect password	System denies access and prompts for a retry.

4.7 Requirements Matrix

Req. ID	Requirement Description	Class	Test Case ID	Status
FR01	Signup: This function allows the user to create a new account.	User	TC01	In Progress
FR02	Login: This function allows the user to log into their existing account.	Human	TC04	In Progress
FR03	Email Validation: This function checks if the email is valid before adding it to the database.	Human	TC02	In Progress
FR04	Password Validation: This function checks if the password is in valid formatting before storing it.	Human	TC03	In Progress
FR05	Word Pacing: This function measures the words spoken per minute and gives appropriate feedback.	Model	TC10-12	Developed
FR06	Grammar Analysis: The system transcribes the audio into text and uses a pre-trained model to give an accurate score.	Model	TC16-17	Developed
FR07	Pronunciation Analysis: The system uses a pre-trained model and passes the corresponding audio file which gives a score for pronunciation.	Model	TC08-09	Developed

Req. ID	Requirement Description	Class	Test Case ID	Status
FR08	Filler word analysis: The system detects filler words and gives appropriate feedback.	Model	TC18	Developed
FR09	Tone Analysis: The system analyses pitch, energy and emotional tone of the speech.	Model	TC13	Developed
FR10	Body Language: The system classifies key points to detect and analyze Posture as well as analyzing facial reactions.	Model	TC14	Developed
FR11	Hand Gestures: The system recognizes hand gestures during speeches and gives feedback.	Model	TC15	Developed
FR12	Upload: The user will have the ability to upload their presentation for analysis.	User	TC06-07	In Progress
FR13	Report Generation: The system generates an overall score for the speech as well as a detailed report for each uploaded session.	Model	TC19	Developed
FR14	History: The user should be able to keep track of previous reports and analysis.	User	TC20	In Progress
FR15	User can request support from the admin.	User	TC15	In Progress

Req. ID	Requirement Description	Class	Test Case ID	Status
FR16	User can delete their account permanently, including all associated data.	User	TC16	In Progress
FR17	Admin can edit user profiles.	Admin	TC17	In Progress
FR18	Admin can assist users in User Support by addressing technical issues.	Admin	TC18	In Progress
FR19	Admin can view user feedback provided by the system.	Admin	TC19	In Progress
FR20	Admin can manage user submissions by reviewing uploaded presentations.	Admin	TC20	In Progress
FR21	Admin can monitor user progress tracking through detailed performance reports.	Admin	TC21	In Progress

4.8 System Deployment

4.8.1 Frameworks

- **Wav2Vec2-Large-Robust-Pronunciation-Evaluation:** A deep learning model that evaluates pronunciation accuracy, helping users refine their speech clarity and articulation.
- **T5-Base-Grammar-Correction (Grammar Analysis):**A transformer-based model that automatically corrects grammatical errors in spoken text.
- **Mediapipe (Gesture Recognition):**Used to track and analyze hand gestures and movements, helping to evaluate expressive gestures during presentations.

- **PoseNet (Posture Analysis):** Detects body posture and alignment, assessing the speaker's stance, stability, and engagement level.
- **Whisper (Speech Recognition):** A state-of-the-art model developed by OpenAI for automatic speech recognition (ASR), used to transcribe speech into text with high accuracy across multiple languages.
- **DeepFace Emotion Recognition Model:** A pre-trained CNN model, fine-tuned for facial emotion detection. It is based on training with the AffectNet and FER-2013 datasets.
- **Flutter Framework**: A platform for building high-performance mobile apps .

4.8.2 Tools

- **OpenCV**: Used for image and video processing, enabling facial expression, body posture, and gesture detection.
- **TensorFlow & Keras**: Provide deep learning capabilities for training and deploying models related to speech, facial expressions, and body language analysis.
- **FFmpeg**: A multimedia processing tool used for handling and converting video/audio files to ensure compatibility and optimal performance.
- **SpeechRecognition Library**: Facilitates speech-to-text conversion, assisting in transcribing spoken words for further analysis.
- **Pydub**: A Python library for audio processing, enabling segmentation, enhancement, and speech pacing analysis.
- **NLTK & spaCy**: Provide natural language processing (NLP) capabilities, including text tokenization, grammar analysis, and sentiment detection.
- **Flask/FastAPI**: Backend frameworks used to develop APIs, ensuring efficient communication between the mobile app and AI models.
- **Firebase**: Handles authentication, real-time database management, and cloud storage for user-submitted videos.

- **Docker**: Provides containerization for consistent deployment, making the system portable and scalable.
- **GitHub**: A version control platform used for collaboration, tracking changes, and managing code repositories.

4.8.3 Technologies

- **Deep Learning**: Utilized for real-time feedback on presentation skills, including speech analysis, facial expression recognition, and gesture evaluation.
- **Natural Language Processing (NLP)**: Used to analyze speech clarity, pacing, tone, grammar, and filler words, providing detailed linguistic feedback.
- **Speech Processing**: Includes automatic speech recognition (ASR) for transcription, pronunciation evaluation, and prosody analysis.
- **Emotion Detection**: Analyzes emotional tone from both voice and facial expressions to assess engagement and confidence levels.
- **Computer Vision**: Enables facial expression detection, body posture assessment, and gesture recognition using pre-trained models.
- **Cloud Computing**: Ensures scalable storage and processing capabilities for AI models and user data, enhancing system accessibility.
- **API Integration**: FastAPI-based backend ensures efficient communication between the mobile application and AI models.
- **Mobile Development**: Built using Flutter, ensuring a seamless and intuitive user experience on both iOS and Android platforms.
- **Data Visualization**: Provides interactive charts and analytics for tracking user progress and feedback.
- **Containerization**: Docker is used to deploy the system in a scalable and portable environment, ensuring consistent performance across platforms.

4.9 Summary

This chapter outlines the system's overall design including: Data,Architecture, and interference viewpoints. It covers key architectural viewpoints:context,logical,composition and interactions. Along with algorithm and interface aspects. The data design section explains dataset structure and storage, while the interface design covers UI elements and user interactions. Implementation insights and testing plans ensure functionality, and deployment strategies define tools and frameworks for performance and maintainability. The chapter concludes with a summary of these aspects, providing a clear understanding of the system's design and development.

Bibliography

[1] ALMUBARAK, A., ALHALABI, W., ALBIDEWI, I., AND ALHARBI, E. An analytical approach for an ai-based teacher performance evaluation system in saudi arabia's schools. *Discover Applied Sciences 6*, 8 (2024), 406.

[2] BAHREINI, K., NADOLSKI, R., AND WESTERA, W. Communication skills training exploiting multimodal emotion recognition. *Interactive Learning Environments 25*, 8 (2017), 1065–1082.

[3] CHEN, J., LAI, P., CHAN, A., MAN, V., AND CHAN, C.-H. Ai-assisted enhancement of student presentation skills: Challenges and opportunities. *Sustainability 15*, 1 (2023).

[4] CHERNER, T., FEGELY, A., HOU, C., AND HALPIN, P. Ai-powered presentation platforms for improving public speaking skills: Takeaways and suggestions for improvement. *Journal of Interactive Learning Research 34*, 2 (2023), 339–367.

[5] DEY, P., AND JANA, D. K. Evaluation of the convincing ability through presentation skills of pre-service management wizards using ai via t2 linguistic fuzzy logic. *Journal of Computational and Cognitive Engineering 2*, 2 (2023), 133–142.

[6] DIMITRIADOU, E., AND LANITIS, A. Evaluating the impact of an automated body language assessment system. *Education and Information Technologies* (8 2024).

[7] EJLOK1. Crema-d. `https://www.kaggle.com/datasets/ejlok1/cremad`, 2025. Accessed: 2025-02-20.

[8] EJLOK1. Surrey audiovisual expressed emotion (savee). https://www.kaggle.com/datasets/ejlok1/surrey-audiovisual-expressed-emotion-savee, 2025. Accessed: 2025-02-20.

[9] EJLOK1. Toronto emotional speech set (tess). https://www.kaggle.com/datasets/ejlok1/toronto-emotional-speech-set-tess, 2025. Accessed: 2025-02-20.

[10] FILLERS, P. Podcast filler dataset. https://podcastfillers.github.io/dataset/, 2025. Accessed: 2025-02-20.

[11] FLINK, N. A., AND COOPER-LARSEN, D. Using an artificial real-time response audience in online sales education to improve self-efficacy in sales presentations: An online classroom innovation. *Atlantic Marketing Journal 9*, 2 (2020), 2.

[12] GUO, R. X. The use of video recordings as an effective tool to improve presentation skills. *Polyglossia 24* (2013), 92–101.

[13] KLAASSEN, R., DE VRIES, P., AND CEULEMANS, D. S. The applicability of feedback of virtual speech app metrics in a presentation technique course sefi 2018 conference, 2018.

[14] MAHOOR, M. Affectnet. http://mohammadmahoor.com/affectnet/, 2025. Accessed: 2025-02-20.

[15] MANSOORIZADEH, M., AND MOGHADDAM CHARKARI, N. Multimodal information fusion application to human emotion recognition from face and speech. *Multimedia Tools and Applications 49*, 2 (2010), 277–297.

[16] MSAMBARE. Fer2013. https://www.kaggle.com/datasets/msambare/fer2013, 2025. Accessed: 2025-02-20.

[17] MULYANI, S., HABIBAH, I. F., MUKAROM, A. M., SUJA, A., ET AL. Leveraging artificial intelligence to develop the presenting skills in arabic language learning. In *International Conference on Education* (2023), pp. 33–46.

[18] OCHOA, X., AND DOMINGUEZ, F. Controlled evaluation of a multimodal system to improve oral presentation skills in a real learning setting. *British Journal of Educational Technology 51*, 5 (2020), 1615–1630.

[19] ORAI. Orai: Ai-powered public speaking coach. `https://orai.com/`, 2025. Accessed: 2025-02-20.

[20] PRIOLO, C. J., AND RE, M. L. L. Ai presentation training software's impact upon students' oral presentation skills. *International Journal of Social Science and Business 7*, 1 (2023).

[21] RUSMIYANTO, R., HURIATI, N., FITRIANI, N., TYAS, N. K., ROFI'I, A., AND SARI, M. N. The role of artificial intelligence (ai) in developing english language learner's communication skills. *Journal on Education 6*, 1 (2023), 750–757.

[22] SAKKALI, A., MEIJBOOM, I., BOGAART, T., AND VAN GINKEL, S. Bled-econference-students-bazaar honest mirror svg.

[23] SHIN, J.-G., CHOI, G.-Y., HWANG, H.-J., AND KIM, S.-H. Evaluation of emotional satisfaction using questionnaires in voice-based human–ai interaction. *Applied Sciences 11*, 4 (2021), 1920.

[24] SPEEKO. Speeko: Ai-powered speech coaching. `https://www.speeko.co/`, 2025. Accessed: 2025-02-20.

[25] TAHERI, M., AND TAN, K. Enhancing presentation skills: A virtual reality-based simulator with integrated generative ai for dynamic pitch presentations and interviews. In *International Conference on Extended Reality* (2024), Springer, pp. 360–366.

[26] TARNOWSKI, P., KOŁODZIEJ, M., MAJKOWSKI, A., AND RAK, R. J. Emotion recognition using facial expressions. *Procedia Computer Science 108* (2017), 1175–1184.

[27] UWRFKAGGLER. Ravdess emotional speech audio. `https://www.kaggle.com/datasets/uwrfkaggler/ravdess-emotional-speech-audio`, 2025. Accessed: 2025-02-20.

[28] VIRTUALSPEECH. Virtualspeech: Virtual reality for presentation training. `https://virtualspeech.com/`, 2025. Accessed: 2025-02-20.

[29] YOUNIS, E. M., MOHSEN, S., HOUSSEIN, E. H., AND IBRAHIM, O. A. S. Machine learning for human emotion recognition: a comprehensive review. *Neural Computing and Applications* (2024), 1–47.

[30] ZOU, B., DU, Y., WANG, Z., CHEN, J., AND ZHANG, W. An investigation into artificial intelligence speech evaluation programs with automatic feedback for developing efl learners' speaking skills. *Sage Open 13*, 3 (2023), 21582440231193818.

Appendix A

Git Repository

Link: `https://github.com/ArwaaMamdoh23/Presentation-Skills.git`

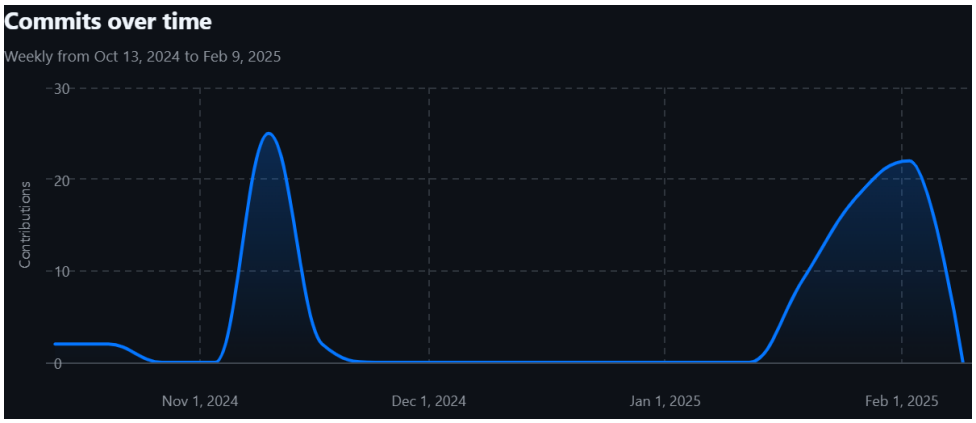

Figure A.1: Git Insights

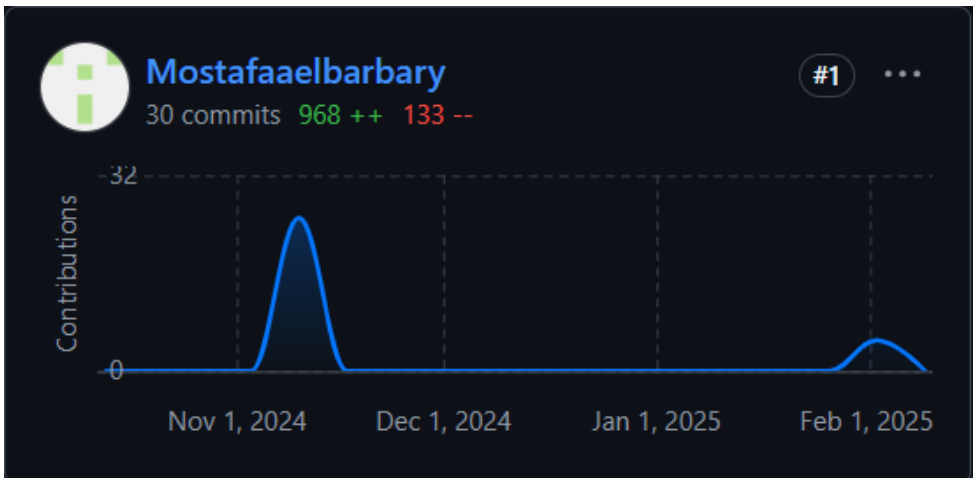

Figure A.2: Git Insights

Figure A.3: Git Insights

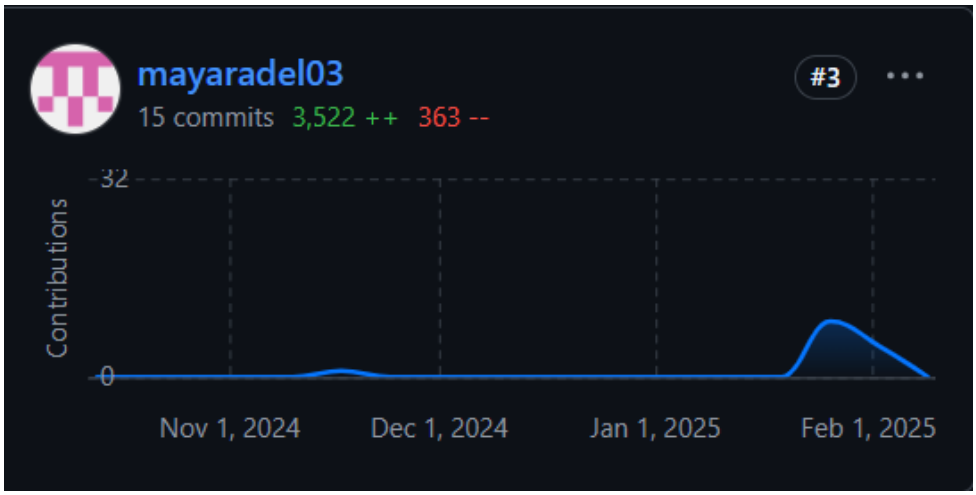

Figure A.4: Git Insights

Figure A.5: Git Insights